

Executors and Asynchronous Operations, Revision 2

1. Introduction

This proposal describes an executors design that uses a lightweight, template-based policy approach. To describe the approach in a nutshell:

An executor is to function execution as an allocator is to allocation.

This proposal is the asynchronous model that underpins the P0112R0 Networking Library proposal, and the proposed wording below is taken from the corresponding sections of P0112R0. In doing so, it takes design concepts from Boost.Asio, many of which have been unchanged since its inclusion in Boost, and repackages them in a way that is more suited to C++14 language facilities.

2. Changes in this revision

In this revision, the proposal has been updated to include only those facilities required by P0112 Networking Library, plus the two classes `thread_pool` and `loop_scheduler`. In particular, this revision incorporates design and wording changes resulting from the LWG wording review in Cologne and the LEWG design review in Lenexa.

3. Reference implementation

A standalone reference implementation of the proposed library can be found at <http://github.com/chriskohlhoff/executors/tree/v0.2-branch>. This implementation requires a C++14 compiler.

To better illustrate how the executors library interacts with asynchronous operations, and to allow the facility to be used in production code, an almost complete implementation of the proposal text may be found in the variant of Asio that stands alone from Boost. This variant is available at <https://github.com/chriskohlhoff/asio/tree/master>.

4. A two minute introduction to the library

Run a function asynchronously:

```
post([]{  
    // ...  
});
```

Run a function asynchronously, on your own thread pool:

```
thread_pool pool;  
  
post(pool, []{  
    // ...  
});
```

```
pool.join();
```

Run a function asynchronously and wait for the result:

```
std::future<int> f =  
    post(use_future([]{  
        // ...  
        return 42;  
    }));
```

```
std::cout << f.get() << std::endl;
```

Run a function asynchronously, on your own thread pool, and wait for the result:

```
thread_pool pool;
```

```
std::future<int> f =
    post(pool, use_future([]{
        // ...
        return 42;
    }));

std::cout << f.get() << std::endl;
```

5. Library vocabulary

The central concept of this library is the *executor* as a policy. An executor embodies a set of rules about where, when and how to run a submitted function object. For example:

Executor type	Executor rules
<code>system_executor</code>	Function objects are allowed to run on any thread in the process.
<code>thread_pool::executor_type</code>	Function objects are allowed to run on any thread in the pool, and nowhere else.
<code>strand<Executor></code>	Run function objects according to the underlying executor's rules, but also run them in FIFO order and not concurrently. Do not block the submitter if the FIFO queue is non-empty.

Executors are ultimately defined by a set of type requirements, so the set of executors isn't limited to those listed here. Like allocators, library users can develop custom executor types to implement their own rules. Executors allow us to encapsulate all sorts of additional information and behaviour on a fine-grained basis, such as:

- Priority.
- Preferred CPU affinity.
- Security credentials or impersonation context.
- How exceptions should be handled.

Certain objects may have an *associated executor*, which specifies how any function objects related to the object should be executed. For example, we may want to say that all event callbacks associated with a network protocol implementation should execute on a particular thread, or that a task should run at a particular priority. The notion of an associated executor allows us to decouple the specification of execution behaviour from the actual point of execution.

An *execution context* is a place where function objects are executed. Where executors are lightweight and cheap to copy, an execution context is typically long-lived and non-copyable. It may contain additional state such as timer queues, socket reactors, or hidden threads to emulate asynchronous functionality. Examples of execution contexts include `thread_pool`, `loop_scheduler`, a Boost.Asio `io_service`, and the set of all threads in the process.

We say that a `thread_pool` is an execution context, and that it *has* an executor. The thread pool contains long-lived state, namely the threads that persist until the pool is shut down. The thread pool's executor embodies the rule: *run functions in the pool and nowhere else*. The thread pool's executor may be obtained by calling its `get_executor()` member function.

To submit a function object to an executor or execution context, we can choose from one of three fundamental operations: *dispatch*, *post* and *defer*. These operations differ in the eagerness with which they run the submitted function.

A dispatch operation is the most eager.

```
dispatch(ex, []{ ... });
```

It means: *block the calling thread until the function completes, if the rules allow it; otherwise, submit for later execution*.

Executor type	Executor rules	Behaviour of dispatch
<code>system_executor</code>	Function objects are allowed to run on any thread in the process.	Always runs the function object before returning from <code>dispatch()</code> .
<code>thread_pool::executor_type</code>	Function objects are allowed to run on any thread in the pool, and nowhere else.	If we are inside the thread pool, runs the function object before returning from <code>dispatch()</code> . Otherwise, adds to the

		thread pool's work queue.
<code>strand<Executor></code>	Run function objects according to the underlying executor's rules, but also run them in FIFO order and not concurrently. Do not block the submitter if the FIFO queue is non-empty.	If we are inside the strand, or if the strand queue is empty, runs the function object before returning from <code>dispatch()</code> . Otherwise, adds to the strand's work queue.

A consequence of calling `dispatch()` is that, if the executor's rules allow it, the compiler is able to inline the function object call.

A post operation, on the other hand, is not permitted to run the function object itself.

```
post(ex, []{ ... });
```

It means: *submit the function for later execution; never block the calling thread to wait for the function to complete*. A posted function is scheduled for execution as soon as possible, according to the rules of the executor.

Executor type	Executor rules	Behaviour of post
<code>system_executor</code>	Function objects are allowed to run on any thread in the process.	Like <code>std::async()</code> , the system executor may allocate <code>std::thread</code> objects to run the submitted function objects. A typical implementation may be to use a hidden system-wide thread pool.
<code>thread_pool::executor_type</code>	Function objects are allowed to run on any thread in the pool, and nowhere else.	Adds the function object to the thread pool's work queue.
<code>strand<Executor></code>	Run function objects according to the underlying executor's rules, but also run them in FIFO order and not concurrently. Do not block the submitter if the FIFO queue is non-empty.	Adds the function object to the strand's work queue.

Finally, the defer operation is the least eager of the three.

```
defer(ex, []{ ... });
```

A defer operation is similar to a post operation, in that it means: *submit the function for later execution; never block the calling thread to wait for the function to complete*. However, a defer operation also *implies a relationship between the caller and the function object being submitted*. It is intended for use when submitting a function object that represents a continuation of the caller.

Executor type	Executor rules	Behaviour of defer
<code>system_executor</code>	Function objects are allowed to run on any thread in the process.	If the caller is executing within the system-wide thread pool, saves the function object to a thread-local queue. Once control returns to the system thread pool, the function object is scheduled for execution as soon as possible. If the caller is not inside the system thread pool, behaves as a post operation.
<code>thread_pool::executor_type</code>	Function objects are allowed to run on any thread in the pool, and nowhere else.	If the caller is executing within the thread pool, saves the function object to a thread-local queue. Once control returns to the thread pool, the function object is scheduled for execution as soon as possible. If the caller is not inside the specified thread pool, behaves as a post operation.
	Run function objects according to the	

<code>strand<Executor></code>	underlying executor's rules, but also run them in FIFO order and not concurrently. Do not block the submitter if the FIFO queue is non-empty.	Adds the function object to the strand's work queue.
-------------------------------------	---	--

6. Library examples

[6.1. Emulating `std::async\(\)`](#)

[6.2. Active objects](#)

[6.3. Activatable objects](#)

[6.4. Leader/Followers pattern](#)

[6.5. Asynchronous operations](#)

[6.6. Pipelines](#)

[6.7. Actors](#)

[6.8. Priority scheduler](#)

In this section we will examine a selection of examples, to see how the proposed executors library supports a range of use cases.

6.1. Emulating `std::async()`

The behaviour of `std::async()` function, when used with `std::launch::async`, may be trivially emulated as follows:

```
template <class F, class... Args>
auto async(F&& f, Args&&... args)
{
    return post(
        use_future(
            std::bind(std::forward<F>(f),
                std::forward<Args>(args)...)));
}
```

Starting from the inside out, the expression:

```
std::bind(std::forward<F>(f),
    std::forward<Args>(args)...)

```

creates a function object that will invoke `f` with the specified arguments. Next:

```
use_future(...)
```

returns an object that will be lazily converted into a function object similar to `std::packaged_task<>`. We could also have used `std::packaged_task<>` directly, as in:

```
std::packaged_task<
    std::result_of_t<
        std::decay_t<F>(std::decay_t<Args>...)>>(>>(...))

```

In this example, the `use_future()` function saves on typing by determining the return type of `F` for us. Finally:

```
post(...)
```

submits the function object for execution on another thread, and then returns immediately. When we submit the result of `use_future()` (or if we were to submit a `std::packaged_task<>`), `post()` automatically deduces its return type to be the `std::future<>` type produced by the task. This future object is then returned from our version of `async()`. Note that, unlike `std::async()`, the returned future object's destructor will not block.

See complete example at:

https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/async_1.cpp

6.2. Active objects

In the Active Object design pattern, all operations associated with an object are run in its own private thread.

To implement an active object, we begin by defining a class member that is a thread pool containing a single thread.

```
class bank_account
```

```
{
    int balance_ = 0;
    mutable thread_pool pool_{1};
    // ...
};
```

We then define each public member function so that it posts its implementation to the thread pool.

```
class bank_account
{
    // ...
    void deposit(int amount)
    {
        post(pool_,
            use_future([=]
                {
                    balance_ += amount;
                }).get());
    }
    // ...
};
```

In more detail, to implement an active object operation we begin by defining the body of the function:

```
[=]
{
    balance_ += amount;
}
```

which we then wrap in a lazily created `std::packaged_task<>` equivalent:

```
use_future(...)
```

Finally, we submit the packaged task to the pool and wait for it to complete. When we submit the result of `use_future()`, `post()` automatically deduces its return type to be the `std::future<>` type produced by the task. We can use this future to block until the operation is complete.

```
post(...).get();
```

See complete example at:

https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/bank_account_2.cpp

6.3. Activatable objects

An Activatable object is a variant of the Active Object pattern where the object does not have a private thread of its own. Instead, it can borrow one of the calling threads to process operations^[1]. However, like Active Object, it ensures that all member state changes do not occur on more than one thread at a time.

To implement an activatable object, we create a strand on the system executor.

```
class bank_account
{
    int balance_ = 0;
    mutable strand<system_executor> strand_;
    // ...
};
```

We then define each public member function so that it dispatches its implementation to the strand.

```
class bank_account
{
    // ...
    void deposit(int amount)
    {
        dispatch(strand_,
            use_future([=]
                {
                    balance_ += amount;
                }).get());
    }
    // ...
};
```

```
};
```

Recall that a `system_executor` object embodies this rule:

Function objects are allowed to run on any thread in the process.

while a strand embodies this rule:

Run function objects according to the underlying executor's rules, but also run them in FIFO order and not concurrently. Do not block the submitter if the FIFO queue is non-empty.

Finally, the call to `dispatch()` means:

Block the calling thread until the function completes, if the rules allow it; otherwise, submit for later execution.

Thus, when we combine `system_executor`, `strand` and `dispatch()`:

```
dispatch(strand_, []{ ... });
```

we are effectively saying: *if the strand is not busy, run the function object immediately*. If there is no contention on the strand, latency is minimised. If there is contention, the strand still ensures that the function object never runs concurrently with any other function object submitted through the same strand.

See complete example at:

https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/bank_account_3.cpp

6.4. Leader/Followers pattern

The Leader/Followers design pattern is a model where multiple threads take turns to wait on event sources in order to dispatch and process incoming events.

Consider an example where a connection handler is responsible for receiving messages from a client via UDP. The Leader/Followers pattern is implemented using a `thread_pool` object:

```
class connection_handler
{
    // ...
private:
    udp_socket socket_;
    thread_pool thread_pool_;
    // ...
};
```

and involves the sequence of operations below.

```
void connection_handler::receive_and_dispatch()
{
```

The leader thread waits for the next message to arrive.

```
    char buffer[1024];
    std::size_t length = socket_.receive(buffer, sizeof(buffer));
```

A new message has arrived. The leader thread promotes a follower to become the new leader.

```
    std::experimental::concurrency::post(thread_pool_,
        [this]{ receive_and_dispatch(); });
```

The now former leader processes the message.

```
    // Process the new message and pass it to the order management bus.
    std::istringstream is(std::string(buffer, length));
    order_management::new_order event;
    if (is >> event)
        order_management_bus_.dispatch_event(event);
}
```

When the function returns, the former leader automatically returns to the pool as a follower thread.

See complete example at:

<https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/trading/server/>

6.5. Asynchronous operations

Asynchronous operations are often chained, and in many cases an object may be associated with two or more chains. For example, an object to manage a connection may contain one chain to do the writing, and another to do the reading:

```
class connection
{
    tcp::socket socket_;
    mutable_buffers_1 in_buffer_;
    mutable_buffers_1 out_buffer_;

    // ...

    void do_read()
    {
        socket_.async_read_some(in_buffer_,
            [this](error_code ec, size_t n)
            {
                // ... process input data ...
                if (!ec) do_read();
            });
    }

    void do_write()
    {
        // ... generate output data ...
        async_write(socket_, out_buffer_,
            [this](error_code ec, size_t n)
            {
                if (!ec) do_write();
            });
    }
};
```

When these chains are run on a single-threaded event loop, it is not possible for more than one completion handler to run at any given time. This means that no synchronisation is required to protected shared data. However, if handlers are executed on a thread pool then some form of synchronisation will be required to avoid introducing data races.

The proposed library provides the `strand<>` template to synchronise handlers. A strand ensures that completion handlers never run concurrently, and explicit synchronisation (such as a mutex) is still not required to protect shared data. To implement this, we use the one strand for all asynchronous operations associated with the object.

```
class connection
{
    tcp::socket socket_;
    mutable_buffers_1 in_buffer_;
    mutable_buffers_1 out_buffer_;
    strand<io_service::executor_type> strand_;

    // ...

    void do_read()
    {
        socket_.async_read_some(in_buffer_,
            bind_executor(strand_, [this](error_code ec, size_t n)
            {
                // ... process input data ...
                if (!ec) do_read();
            }));
    }

    void do_write()
    {
        // ... generate output data ...
        async_write(socket_, out_buffer_,
            bind_executor(strand_, [this](error_code ec, size_t n)
            {
                if (!ec) do_write();
            }));
    }
};
```

```
};
```

The `bind_executor` function is used to associate an executor with an object. In this example, we used `bind_executor` to associate the strand with each of the lambdas. The `bind_executor` function works with any executor or execution context. For example, here we associate a thread pool with a lambda:

```
async_getline(std::cin,
    bind_executor(pool, [](std::string line)
    {
        std::cout << "Line: " << line << "\n";
    }));
```

Rather than using the `bind_executor` function, the associated executor may be manually specified by providing a nested `executor_type` typedef and `get_executor()` member function.

```
class line_printer
{
public:
    typedef loop_scheduler::executor_type executor_type;

    explicit line_printer(loop_scheduler& s)
        : executor_(s.get_executor())
    {
    }

    executor_type get_executor() const noexcept
    {
        return executor_;
    }

    void operator()(std::string line)
    {
        std::cout << "Line: " << line << "\n";
    }

private:
    loop_scheduler::executor_type executor_;
};

// ...

async_getline(std::cin, line_printer(scheduler));
```

For this to work correctly, the `async_getline` asynchronous operation must participate in an executor-aware model. To be executor-aware, an asynchronous operation must:

- Ask the completion handler for its associated executor, by calling `get_associated_executor`.
- While pending, maintain an `executor_work_guard` object for the associated executor. This tells the executor to expect a function object to be submitted in the future. A thread pool, for example, will know that it still has work to do and needs to keep running.
- Dispatch, post or defer any intermediate handlers, and the final completion handler, through the associated executor. This ensures that all handlers are executed according to the executors rules.

Our `async_getline` operation can then be written as follows:

```
template <class Handler>
void async_getline(std::istream& is, Handler handler)
{
```

The `make_work_guard` function automatically obtains the associated executor and creates an `executor_work_guard` object for it.

```
    auto work = make_work_guard(handler);
```

The asynchronous operation itself is posted outside of the associated executor. This is because we want the line reading to be performed asynchronously with respect to the caller.

```
    post([&is, work, handler=std::move(handler)]() mutable
    {
        std::string line;
        std::getline(is, line);
```

Once the asynchronous work is complete, we execute the completion handler via its associated executor.

```
// Pass the result to the handler, via the associated executor.
dispatch(work.get_executor(),
    [line=std::move(line), handler=std::move(handler)]() mutable
    {
        handler(std::move(line));
    });
});
}
```

See complete example at:

https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/async_op_1.cpp

When composing asynchronous operations, intermediate operations can simply reuse the associated executor of the final handler.

```
template <class Handler>
void async_getlines(std::istream& is, std::string init, Handler handler)
{
    // Get the final handler's associated executor.
    auto ex = get_associated_executor(handler);

    // Use the associated executor for each operation in the composition.
    async_getline(is,
        bind_executor(ex, [&is, lines=std::move(init), handler=std::move(handler)]
            (std::string line) mutable
            {
                if (line.empty())
                    handler(lines);
                else
                    async_getlines(is, lines + line + "\n", std::move(handler));
            });
    });
}
```

This ensures that all intermediate completion handlers are correctly executed according to the caller's executor's rules.

See complete example at:

https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/async_op_2.cpp

6.6. Pipelines

A pipeline is a sequence of two or more long-running functions, known as stages, with each stage passing data to the next via a queue. The initial stage acts as a source of data, the intermediate stages act as filters, and the final stage as a sink.

As an example, let us consider a small framework for implementing pipelines. The function used to construct a pipeline is declared as:

```
template <class F, class... Tail>
std::future<void> pipeline(F f, Tail... t);
```

This function returns a future that can be used to wait until the pipeline is complete. The initial stage of the pipeline must be a function with signature:

```
void initial(queue_front<T0> out);
```

The intermediate stages have signature:

```
void intermediate(queue_back<Tn> in, queue_front<Tn+1> out);
```

The pipeline's final stage has the signature:

```
void final(queue_back<TN> in);
```

By default, we want each stage of a pipeline to have its own thread. The pipeline framework achieves this by calling `get_associated_executor` with two arguments:

```
template <class F, class... Tail>
std::future<void> pipeline(F f, Tail... t)
```

```
{
    // ...
    auto ex = get_associated_executor(f, thread_executor());
    // ...
}
```

The `thread_executor` class is a custom executor type defined for the example. It starts a new thread for every function passed to `dispatch`, `post` or `defer`. If the function object type `F` already has an associated executor then that executor will be used. The `thread_executor` is used for types that do not specify an associated executor.

So, when we construct and run a pipeline like this:

```
void reader(queue_front<std::string> out);
void filter(queue_back<std::string> in, queue_front<std::string> out);
void upper(queue_back<std::string> in, queue_front<std::string> out);
void writer(queue_back<std::string> in);

// ...

thread_pool pool;
auto f = pipeline(reader, filter, bind_executor(pool, upper), writer);
f.wait();
```

we are specifying that the the upper stage should run on the thread pool, while reader, filter and writer should use the default behaviour of launching a new thread.

See complete example at:

<https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/pipeline.cpp>

6.7. Actors

The Actor model is a model for concurrency where objects, known as actors, communicate only by sending and receiving messages. Each actor's state is accessed only by its own internal thread or strand. This means that actors are inherently thread-safe.

To illustrate how executors may be used to facilitate actors, a tiny actor framework is included with the executors reference implementation. This framework is loosely based on the Theron library^[2].

To implement an actor using this framework, we start by deriving a class from `actor`:

```
class member : public actor
{
    // ...
}
```

When constructing an actor, we specify the executor to be used:

```
explicit member(executor e)
    : actor(std::move(e))
{
    // ...
}
```

The polymorphic type `executor` is used to allow the selection of an actor's executor to be delayed until runtime. All of an actor's message handlers are executed according to that policy. This could be a thread pool executor, but we may equally construct actors with an executor that [knows about priorities](#).

The actor's message handlers are member functions, identified by argument type, and may be arbitrarily registered or deregistered:

```
void init_handler(actor_address next, actor_address from)
{
    // ...
    register_handler(&member::token_handler);
    deregister_handler(&member::init_handler);
}
```

Internally, the actor framework uses a per-actor strand to ensure that the member functions are never called concurrently.

To send a message between actors we use either `actor::send()` or `actor::tail_send()`. The `send()` operation is

implemented in terms of the actor's executor's `post()` member function. The `tail_send()` function is a distinct operation and conveys additional information about the caller's intent which may be used to optimise inter-actor messaging. It is implemented in terms of `defer()`.

```
void token_handler(int token, actor_address /*from*/)
{
    // ...
    tail_send(msg, to);
}

// ...
};
```

See complete example at:

<https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/actor.cpp>

6.8. Priority scheduler

Executor objects are lightweight and copyable to allow us to encapsulate all sorts of additional information and behaviour on a fine-grained basis. One use case for this is attaching priorities to function objects or tasks.

We begin by defining our priority scheduler class as an execution context. Internally, this class uses a priority queue to store pending function objects.

```
class priority_scheduler : public execution_context
{
    // ...

private:

    // ...

    struct item_comp
    {
        bool operator()(
            const std::shared_ptr<item_base>& a,
            const std::shared_ptr<item_base>& b)
        {
            return a->priority_ < b->priority_;
        }
    };

    std::mutex mutex_;
    std::condition_variable condition_;
    std::priority_queue<
        std::shared_ptr<item_base>,
        std::vector<std::shared_ptr<item_base>>,
        item_comp> queue_;
    bool stopped_ = false;
};
```

The `priority_scheduler` class provides a nested class `executor_type` which satisfies the executor type requirements, and a member function `get_executor()` to obtain an executor object. On construction, an `executor_type` object captures a reference to the priority scheduler, as well as the specified priority value.

```
class priority_scheduler : public execution_context
{
public:
    class executor_type
    {
    public:
        executor_type(priority_scheduler& ctx, int pri) noexcept
            : context_(ctx), priority_(pri)
        {
        }

        // ...

    private:
        priority_scheduler& context_;
```

```

    int priority_;
};

executor_type get_executor(int pri = 0) noexcept
{
    return executor_type(*this, pri);
}

// ...
};

```

When a function object is submitted, the executor uses its stored priority to insert the function into the correct position in the priority queue:

```

class priority_scheduler : public execution_context
{
public:
    class executor_type
    {
    public:

        // ...

        template <class Func, class Alloc>
        void post(Func f, const Alloc& a)
        {
            auto p(std::allocate_shared<item<Func>>(a, priority_, std::move(f)));
            std::lock_guard<std::mutex> lock(context_.mutex_);
            context_.queue_.push(p);
            context_.condition_.notify_one();
        }

        // ...
    };

    // ...
};

```

The priority scheduler's executors can then be used like any other:

```

priority_scheduler sched;
auto low = sched.get_executor(0);
auto med = sched.get_executor(1);
auto high = sched.get_executor(2);
// ...
dispatch(low, []{ std::cout << "1\n"; });
dispatch(med, []{ std::cout << "2\n"; });
dispatch(high, []{ std::cout << "3\n"; });

```

See complete example at:

https://github.com/chriskohlhoff/executors/blob/v0.2-branch/src/examples/executor/priority_scheduler.cpp

7. Summary of library facilities

Header	Name	Description
executor	Class template <code>async_result</code>	Determines the result of an asynchronous operation's initiating function.
executor	Class template <code>async_completion</code>	Helper to simplify implementation of an asynchronous operation.
executor	Class template <code>associated_allocator</code>	Used to determine a handler's associated allocator.
executor	Function <code>get_associated_allocator</code>	Obtain a handler's associated allocator.
executor	Class template <code>execution_context</code>	Base class for execution context types.

executor	Class template associated_executor	Used to determine a handler's associated executor.
executor	Function get_associated_executor	Obtain a handler's associated executor.
executor	Class template executor_binder	Associates an executor with an object.
executor	Function bind_executor	Associate an executor with an object.
executor	Class template executor_work_guard	Tracks outstanding work against an executor.
executor	Function make_work_guard	Create work to track an outstanding operation.
executor	Class system_executor	Executor representing all threads in system.
executor	Class system_context	The execution context underlying system_executor instances.
executor	Class executor	Polymorphic wrapper for executors.
executor	Functions dispatch, post and defer	Execute a function object.
executor	Class template strand	Executor adapter that runs function objects non-concurrently, in FIFO order, and without blocking the submitter if the FIFO queue is non-empty.
executor	Class template use_future_t	Completion token to enable futures with asynchronous operations.
executor	Class template specialization of async_result for packaged_task	Supports use of packaged_task with dispatch, post, defer, and asynchronous operations.
thread_pool	Class thread_pool	A fixed size thread pool.
loop_scheduler	Class loop_scheduler	A thread pool where threads are explicitly donated by the caller.

8. On the naming of executors

There has been some confusion due to the reuse of the term *executor* in N3785 and successors and this proposal, but with slightly different meanings. In N3785, an "executor" refers to a heavyweight, non-copyable object, such as a thread pool. In this proposal, an "executor" is a lightweight, copyable policy object. This is distinct from a heavyweight object such as a thread pool, which is known as an "execution context".

N3785's API is superficially similar to Java executors, so it is interesting to examine the Java prior art in this area. What we find is that this approach misses a key concept: the separation of *Executor* and *ExecutorService*. On the other hand, it turns out that this proposal's "executor" mirrors the concept *and* terminology of Java executors.

Let us start by reviewing a couple of the core interfaces of the Java executor framework: <http://docs.oracle.com/javase/7/docs/api/java/util/concurrent/package-summary.html>

First, we have interface *Executor*. This interface provides a way of submitting a *Runnable* (i.e. the equivalent of a function object) for execution, and it decouples the submission from the concrete mechanism which runs the function.

Second, we have interface *ExecutorService*. This extends *Executor*, i.e. *ExecutorService is-a Executor*. It adds some additional functionality, such as the ability to request that it be shut down.

A thread pool *is-a ExecutorService*. A fork/join pool *is-a ExecutorService*. An *ExecutorService* represents a heavyweight entity where *Runnable* items are run.

A *SerialExecutor is-a Executor*. A thread-per-task executor *is-a Executor*. An executor represents a policy. Where there is a customisation point (as in the *ExecutorCompletionService* class) it is specified as an *Executor*.

When we want to create our own policy, we do it by implementing the *Executor* interface. Our *Executor* policy object can be short lived, or it can be long lived. As we are using Java, the object is newed and we let the garbage collector take

care of it. In fact, we don't really have a choice.

Java is not C++. Java references are not C++ references, nor are they C++ pointers. We do not have the garbage collector to clean up after us. Yet, and this is especially true of concurrent code, correctly managing object lifetime is critical. How do we address this in C++? The idiomatic approach is to use value semantics.

Thus our Executor policy object should use value semantics. We should be able to copy it and move it freely. Where a particular concrete Executor uses some allocated resource, the constructors and destructors can manage the resource lifetime, just as we do in other standard library components.

Of course, we do want to be able to use a heavyweight thread pool as an Executor. In Java, the thread pool *is-a* ExecutorService which *is-a* Executor, so we are able to use the heavyweight object in the same way as a lightweight one. Once again, this is because of Java's reference semantics, where basically all objects are treated the same, whether light or heavy.

In C++, however, our heavyweight thread pool may be best represented by a non-copyable type. This presents a challenge: how do we establish a pattern where we can pass either a noncopyable type or a type with value semantics? That is, how can we have an interface where we can pass either a heavyweight ExecutorService or a lightweight Executor?

The solution is to change the relationship between ExecutorService and Executor. Rather than saying an ExecutorService *is-a* Executor, we instead say an ExecutorService **has-a** Executor. Every ExecutorService has an associated lightweight Executor policy which encapsulates the submission logic. This lightweight Executor provides the necessary value semantics.

Thus we can see that this proposal's "executor" is in fact the same concept as the Java Executor. It is just that it is packaged in a way that is more idiomatic C++, i.e. it uses value semantics. This proposal's "execution context" concept is the equivalent of Java's ExecutorService. The executor is the "how", a policy, and it logically performs the execution. The execution context or ExecutorService is the "where", a venue if you like.

As we are using C++, and not Java, we get the same level of abstraction but with the benefits of compile time polymorphism, inlining, control over memory use, and using fewer allocations (i.e. we create less garbage). The reasons we are using C++ in the first place.

As we can see, the separation of executor from execution context clearly exists in the prior art represented by Java. However, this proposal's design is derived from Boost.Asio, and is very much driven by what is required to make asynchronous operations work, but with a desire to have a clean separation of concerns. The Java executors framework did not inform the design, yet it is not surprising that Java represents an example of convergent evolution, once we make a deeper analysis of the library.

9. On the need for dispatch, post and defer

Let us take another look at the specification of the Java Executor class and its execute method: <http://docs.oracle.com/javase/7/docs/api/java/util/concurrent/Executor.html>

It says:

```
void execute(Runnable command)
```

Executes the given command at some time in the future. The command may execute in a new thread, in a pooled thread, or in the calling thread, at the discretion of the Executor implementation.

Note that the wording includes "in the calling thread". In fact, the specification of the execute method is essentially the same as the dispatch function of this proposal.

As it is the only available Executor method, this specification of execute is a problem. Clearly there are times when we want to guarantee that a function will *not* run in the calling thread, such as when we want launch a long running function. Unfortunately, if we are using an Executor in a polymorphic context we have no way of knowing what behaviour we will get. We have to know the concrete executor type to ensure that it doesn't run in the calling thread.

Thus, we wish to introduce a function with slightly different semantics:

Executes the given command at some time in the future. The command may execute in a new thread, in a pooled thread, at the discretion of the Executor implementation.

This is the post function of this proposal, and it lets us as the caller ensure that a function does not run in the calling

thread.

However, this does not obviate the need for the original semantics (i.e. the current Java semantics). There are times when it is to our advantage to allow a function to run in the calling thread. Some examples:

- When using a strand (or a serial executor, in Java terminology) to ensure non-concurrent function execution. If the strand is not contended, we want to execute the function in the current thread. This will minimise latency and avoid the cost of a context switch.
- At the end of an asynchronous operation. We are unwrapping layers of composition and handing the result back through a stack of callbacks. A callback needs to run on its correct executor, but we do not want to unnecessarily incur a context switch or a cycle through the scheduler, as doing so significantly adds to the latency in processing the event.

Interestingly, the specification of `future::then` (now removed from the concurrency TS) that took an executor would also suffer from this extra cost unless it had access to dispatch semantics.

The `defer` operation, like `post`, does not allow the function to run in the calling thread. Where it differs is in the expression of the intent of the caller. Using `defer` states that there is a relationship between the caller and callee, such as being consecutive links in a chain of function objects. An executor can make use of this to do smarter, more efficient scheduling.

For example, consider a chain of asynchronous read operations on a socket:

```
void read_loop(Socket socket, Buffer buffer)
{
    async_read(socket, buffer,
        [&](error_code, size_t n) {
            process_data(buffer, n);
            read_loop(socket, buffer);
        });
}
```

where an individual read operation is implemented something like this:

```
template <class Handler>
void async_read(Socket socket, Buffer buffer, Handler handler)
{
    // Perform a speculative read first.
    error_code ec;
    size_t n = non_blocking_read(socket, buffer, ec);
    if (ec != would_block)
    {
        // Read completed immediately, post handler.
        ex = get_associated_executor(handler);
        post(ex, [=]{ handler(ec, n); });
    }
    else
    {
        // Wait for socket to become readable.
        // ...
    }
}
```

In certain circumstances the read operation will always complete immediately, such as when the data is already available in the kernel buffers. When this occurs, the sequence of operations is essentially equivalent to:

```
void read_loop(socket, buffer)
{
    // ...
    ex.post([&]{ // #1
        read_loop(socket, buffer);
    });
    // ...
}
```

Let us assume that our executor `ex` uses a thread pool with a mutex-protected queue:

```
class my_thread_pool
{
public:
```

```

class executor_type
{
public:
    // ...

    template <class Func, class Alloc>
    void post(Func f, const Alloc& a)
    {
        auto p(std::allocate_shared<item<Func>>(a, std::move(f)));
        std::lock_guard<std::mutex> lock(pool_.mutex_); // #2
        pool_.queue_.push_back(std::move(p)); // #3
        pool_.condition_.notify_one(); // #4
        // #5
    }

    // ...
};

// ...

void run()
{
    for (;;)
    {
        std::unique_lock<std::mutex> lock(mutex_); // #6
        condition_.wait(lock, [&]{ !queue_.empty(); });
        auto p(std::move(queue_.front())); // #7
        queue_.pop_front();
        lock.unlock(); // #8
        p->execute_(p); // #9
    }
}

private:
    std::mutex mutex_;
    std::condition_variable condition_;
    std::deque<std::shared_ptr<item_base>> queue_;
};

```

There are two performance issues at play here. First, each "cycle" of `read_loop` involves two lock/unlock pairs. Second, a condition variable may be used to wake a sleeping thread when the queue is non-empty. If we step through the code we will see the following:

- [#6](#) — lock
- [#7](#) — dequeue `read_loop`
- [#8](#) — unlock
- [#9](#) — call `read_loop`
 - [#1](#) — call `post`
 - [#2](#) — lock
 - [#3](#) — enqueue `read_loop`
 - [#4](#) — notify
 - [#5](#) — unlock
- *(start of next cycle)*
- [#6](#) — lock
- [#7](#) — dequeue `read_loop`
- [#8](#) — unlock
- [#9](#) — call `read_loop`
- ...

On the other hand, with `defer` we are telling the executor that the submitted function is a continuation of the current one. That is, the executor does not have to eagerly schedule the function because we have told it that one function follows the other.

```

void read_loop(socket, buffer)
{
    // ...
    ex.defer([&]{ // #1
        read_loop(socket, buffer);
    });
}

```

```

    });
    // ...
}

class my_thread_pool
{
public:
    class executor_type
    {
    public:
        // ...

        template <class Func, class Alloc>
        void defer(Func f, const Alloc& a)
        {
            if (pool_.thread_local_queue_)
            {
                auto p(std::allocate_shared<item<Func>>(a, std::move(f)));
                pool_.thread_local_queue_>push_back(std::move(p)); // #2
            }
            else
                post(std::move(f), a);
        }

        // ...
    };

    // ...

    void run()
    {
        std::deque<std::shared_ptr<item_base>> local_queue;
        thread_local_queue_ = &local_queue;
        for (;;)
        {
            std::unique_lock<std::mutex> lock(mutex_); // #3
            while (!local_queue.empty()) // #4
            {
                queue_.push(std::move(local_queue.front()));
                local_queue.pop_front();
            }
            condition_.wait(lock, [&]{ !queue_.empty(); });
            auto p(std::move(queue_.front())); // #5
            queue_.pop_front();
            lock.unlock(); // #6
            p->execute_(p); // #7
        }
    }

private:
    std::mutex mutex_;
    std::condition_variable condition_;
    std::deque<std::shared_ptr<item_base>> queue_;
    static thread_local std::deque<std::shared_ptr<item_base>>* thread_local_queue_;
};

```

Now when we step through the code:

- [#3](#) — lock
- [#4](#) — copy contents of thread-local queue to main queue
- [#5](#) — dequeue read_loop
- [#6](#) — unlock
- [#7](#) — call read_loop
 - [#1](#) — call defer
 - [#2](#) — enqueue read_loop to thread-local queue
- *(start of next cycle)*
- [#3](#) — lock
- [#4](#) — copy contents of thread-local queue to main queue
- [#5](#) — dequeue read_loop

- [#6](#) — unlock
- [#7](#) — call read_loop
- ...

we see that we have eliminated one lock/unlock pair, and we also no longer need to wake another thread. We are able to do this because of the additional information imparted by defer.

On recent hardware we can observe an uncontended lock/unlock cost of some 10 to 15 nanoseconds, compared with 1 to 2 nanoseconds for accessing a thread-local queue. There is also a significant (and often larger) benefit in avoiding the unnecessary thread wakeup and the ensuing lock contention, particularly when dealing with bursty traffic profiles. Either way, this is a latency win.

With asynchronous operations, the rules are that if an operation completes immediately it posts the result (rather than dispatch, which may result in unfairness, starvation or stack overflow). If it finishes later, it dispatches the result (to minimise latency).

By default, an individual low-level asynchronous operation, such as `async_read` shown above, doesn't know if the operation represents a continuation of the current function, or a new fork in the control flow. Either one is possible, so we conservatively assume that every operation represents a new fork and use `post`.

However, once we move to a higher layer of abstraction, like a composed operation to read a message frame, we can start to make certain assertions. We know that within the operation it consists of a single chain of asynchronous reads.

As an example, let us consider a hypothetical composed operation to read a message frame, implemented in terms of `async_read` above. Each message frame consists of a header, a body in several chunks, and a trailer. In this scenario, the header and body are immediately available in the kernel buffers, but we have to wait for the trailer to arrive. The sequence of executor operations used by the asynchronous chain looks like this:

- header available immediately → `post()`
- first body chunk available immediately → `post()`
- second body chunk available immediately → `post()`
- ... *waiting for trailer* ...
- trailer available → `dispatch()`

One of the motivating reasons for having lightweight, copyable executors, distinct from the execution context, is that they let us remap the executor operations as required. Thus, within the composed operation we can remap `post` to `defer`. We can do this with a lightweight wrapper around the composed operation's handler's associated executor:

```
template <class Executor>
class remap_post_to_defer
{
    ...
    template <class F, class A>
    void post(F f, const A& a)
    {
        ex_.defer(std::move(f), a);
    }
    ...
    Executor ex_;
};
```

We can then apply this wrapper to optimise the intermediate steps of the chain:

- header available immediately → `post()`
- first body chunk available immediately → `defer()`
- second body chunk available immediately → `defer()`
- ... *waiting for trailer* ...
- trailer available → `dispatch()`

If we had a limited vocabulary that only consisted of `dispatch`:

- header available immediately → `dispatch()`
- first body chunk available immediately → `dispatch()`
- second body chunk available immediately → `dispatch()`
- ... *waiting for trailer* ...
- trailer available → `dispatch()`

then traffic bursts can lead to unfairness and starvation. We are susceptible to denial of service attacks.

If our vocabulary only consisted of `post`:

- header available immediately → `post()`
- first body chunk available immediately → `post()`
- second body chunk available immediately → `post()`
- ... *waiting for trailer* ...
- trailer available → `post()`

then every operation in the chain can incur otherwise avoidable synchronisation costs, context switches, and cycles through the scheduler, resulting in higher latency.

If our vocabulary only consisted of `defer`:

- header available immediately → `defer()`
- first body chunk available immediately → `defer()`
- second body chunk available immediately → `defer()`
- ... *waiting for trailer* ...
- trailer available → `defer()`

then we almost get away with it, apart from the additional latency introduced by `defer` at the end of the operation. However, we are also limiting the opportunities for concurrency. This may not be an issue in this example with a single chain of operations, but can be a problem where your asynchronous control flow really does fork, such as in a typical accept "loop":

```
void handle_accept()
{
    new_socket->start(); // starts asynchronous reads and writes
    async_accept(..., &handle_accept); // accepts next connection
}
```

Thus we need all three operations to complete the set:

- `post` — the default choice, guaranteeing non-blocking calls and maximising concurrency
- `dispatch` — for minimising latency when we are prepared to accept blocking
- `defer` — to link sequences of related operations

Note that, when implementing the executor type requirements, it is perfectly fine to start by implementing `dispatch` and `defer` in terms of `post`. This is in keeping with the specified semantics. Then, we can optimise the implementation of these functions as we are able to.

However, are these three operations sufficient? Might there be more things that a user wants to communicate to the executor, about how a function or task should be launched? For example, a priority or a hard real-time deadline.

The proposed library meets these needs by giving a function object or task an associated executor. As lightweight, copyable objects, executors allow us to encapsulate all sorts of additional information and behaviour on a fine-grained basis, such as priority. The associated executor determines how it should be executed, and the point of association may be distant in time and space from the point where a function is submitted using `dispatch`, `post` and `defer`.

10. Impact on the standard

This is a pure library proposal. It does not add any new language features, nor does it alter any existing standard library headers. It makes additions to experimental headers that may also be modified by other Technical Specifications.

This library can be implemented using compilers that conform to the C++14 standard. An implementation of this library requires operating system-specific functions that lie outside the C++14 standard.

11. Relationship to other proposals

This proposal specifies the asynchronous model that underpins the P0112R0 Networking Library proposal. The proposed wording below is taken from the corresponding sections of P0112R0, but under a different namespace.

12. Proposed text

[12.1. Library summary](#)

[12.2. Header <experimental/executor> synopsis](#)

[12.3. Requirements](#)

[12.3.1. Proto-allocator requirements](#)

[12.3.2. Execution context requirements](#)

[12.3.3. Executor requirements](#)

[12.3.4. Service requirements](#)

[12.3.5. Signature requirements](#)

[12.3.6. Associator requirements](#)

[12.3.7. Requirements on asynchronous operations](#)

[12.3.7.1. General asynchronous operation concepts](#)

[12.3.7.2. Completion tokens and handlers](#)

[12.3.7.3. Automatic deduction of initiating function return type](#)

[12.3.7.4. Production of initiating function return value](#)

[12.3.7.5. Lifetime of initiating function arguments](#)

[12.3.7.6. Non-blocking requirements on initiating functions](#)

[12.3.7.7. Associated executor](#)

[12.3.7.8. I/O executor](#)

[12.3.7.9. Completion handler executor](#)

[12.3.7.10. Outstanding work](#)

[12.3.7.11. Allocation of intermediate storage](#)

[12.3.7.12. Execution of completion handler on completion of asynchronous operation](#)

[12.3.7.13. Completion handlers and exceptions](#)

[12.4. Class template `async\_result`](#)

[12.5. Class template `async\_completion`](#)

[12.6. Class template `associated\_allocator`](#)

[12.6.1. `associated\_allocator` members](#)

[12.7. Function `get\_associated\_allocator`](#)

[12.8. Class `execution\_context`](#)

[12.8.1. `execution\_context` constructor](#)

[12.8.2. `execution\_context` destructor](#)

[12.8.3. `execution\_context` operations](#)

[12.8.4. `execution\_context` protected operations](#)

[12.8.5. `execution\_context` globals](#)

[12.9. Class `execution\_context::service`](#)

[12.10. Class template `is\_executor`](#)

[12.11. Executor argument tag](#)

[12.12. `uses\_executor`](#)

[12.12.1. `uses\_executor` trait](#)

[12.12.2. `uses\_executor` construction](#)

[12.13. Class template `associated\_executor`](#)

[12.13.1. `associated\_executor` members](#)

[12.14. Function `get\_associated\_executor`](#)

[12.15. Class template `executor\_binder`](#)

[12.15.1. `executor\_binder` constructors](#)

[12.15.2. `executor\_binder` access](#)

[12.15.3. `executor\_binder` invocation](#)

[12.15.4. Class template partial specialization `async\_result`](#)

[12.15.5. Class template partial specialization `associated\_allocator`](#)

[12.15.6. Class template partial specialization `associated\_executor`](#)

[12.16. Function `bind\_executor`](#)

[12.17. Class template `executor\_work\_guard`](#)

[12.17.1. `executor\_work\_guard` members](#)

[12.18. Function `make\_work\_guard`](#)

[12.19. Class `system\_executor`](#)

[12.19.1. `system\_executor` operations](#)

[12.19.2. `system\_executor` comparisons](#)

[12.20. Class `system\_context`](#)

[12.21. Class `bad\_executor`](#)

[12.22. Class `executor`](#)

[12.22.1. `executor` constructors](#)

[12.22.2. `executor` assignment](#)

- [12.22.3. executor destructor](#)
- [12.22.4. executor modifiers](#)
- [12.22.5. executor operations](#)
- [12.22.6. executor capacity](#)
- [12.22.7. executor target access](#)
- [12.22.8. executor comparisons](#)
- [12.22.9. executor specialized algorithms](#)
- [12.23. Function dispatch](#)
- [12.24. Function post](#)
- [12.25. Function defer](#)
- [12.26. Class template strand](#)
 - [12.26.1. strand constructors](#)
 - [12.26.2. strand assignment](#)
 - [12.26.3. strand destructor](#)
 - [12.26.4. strand operations](#)
 - [12.26.5. strand comparisons](#)
- [12.27. Class template use_future_t](#)
 - [12.27.1. use_future_t constructors](#)
 - [12.27.2. use_future_t members](#)
 - [12.27.3. Partial class template specialization async_result for use_future_t](#)
- [12.28. Partial class template specialization async_result for packaged_task](#)
- [12.29. Header <experimental/thread_pool> synopsis](#)
- [12.30. Class thread_pool](#)
 - [12.30.1. thread_pool members](#)
- [12.31. Class thread_pool::executor_type](#)
 - [12.31.1. thread_pool::executor_type constructors](#)
 - [12.31.2. thread_pool::executor_type assignment](#)
 - [12.31.3. thread_pool::executor_type operations](#)
 - [12.31.4. thread_pool::executor_type comparisons](#)
- [12.32. Header <experimental/loop_scheduler> synopsis](#)
- [12.33. Class loop_scheduler](#)
 - [12.33.1. loop_scheduler members](#)
- [12.34. Class loop_scheduler::executor_type](#)
 - [12.34.1. loop_scheduler::executor_type constructors](#)
 - [12.34.2. loop_scheduler::executor_type assignment](#)
 - [12.34.3. loop_scheduler::executor_type operations](#)
 - [12.34.4. loop_scheduler::executor_type comparisons](#)

12.1. Library summary

Table 1. Library summary

Clause	Header(s)
Asynchronous model	<experimental/executor>
A fixed-size thread pool	<experimental/thread_pool>
A thread pool where existing threads are assigned to the pool	<experimental/loop_scheduler>

Throughout this Technical Specification, the names of the template parameters are used to express type requirements, as listed in the table below.

Table 2. Template parameters and type requirements

template parameter name	type requirements
Allocator	C++Std [allocator.requirements]
Clock	C++Std [time.clock.req]
CompletionToken	completion token

ExecutionContext	execution context
Executor	executor
ProtoAllocator	proto-allocator
Service	service
Signature	signature

12.2. Header <experimental/executor> synopsis

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class CompletionToken, class Signature, class = void>
        class async\_result;

    template<class CompletionToken, class Signature>
        struct async\_completion;

    template<class T, class ProtoAllocator = allocator<void>>
        struct associated\_allocator;

    template<class T, class ProtoAllocator = allocator<void>>
        using associated_allocator_t = typename associated_allocator<T, ProtoAllocator>::type;

    // get\_associated\_allocator:

    template<class T>
        associated_allocator_t<T> get_associated_allocator(const T& t) noexcept;
    template<class T, class ProtoAllocator>
        associated_allocator_t<T, ProtoAllocator>
            get_associated_allocator(const T& t, const ProtoAllocator& a) noexcept;

    enum class fork_event {
        prepare,
        parent,
        child
    };

    class execution\_context;

    class service_already_exists;

    template<class Service> Service& use_service(execution_context& ctx);
    template<class Service, class... Args> Service&
        make_service(execution_context& ctx, Args&&... args);
    template<class Service> bool has_service(execution_context& ctx) noexcept;

    template<class T> struct is\_executor;

    struct executor\_arg\_t { };
    constexpr executor_arg_t executor_arg = executor_arg_t();

    template<class T, class Executor> struct uses\_executor;

    template<class T, class Executor = system_executor>
        struct associated\_executor;

    template<class T, class Executor = system_executor>
        using associated_executor_t = typename associated_executor<T, Executor>::type;

    // get\_associated\_executor:

    template<class T>
        associated_executor_t<T> get_associated_executor(const T& t) noexcept;
    template<class T, class Executor>

```

```

    associated_executor_t<T, Executor>
        get_associated_executor(const T& t, const Executor& ex) noexcept;
template<class T, class ExecutionContext>
    associated_executor_t<T, typename ExecutionContext::executor_type>
        get_associated_executor(const T& t, ExecutionContext& ctx) noexcept;

template<class T, class Executor>
    class executor\_binder;

template<class T, class Executor, class Signature>
    class async_result<executor_binder<T, Executor>, Signature>;

template<class T, class Executor, class ProtoAllocator>
    struct associated_allocator<executor_binder<T, Executor>, ProtoAllocator>;

template<class T, class Executor, class Executor1>
    struct associated_executor<executor_binder<T, Executor>, Executor1>;

// bind\_executor:

template<class Executor, class T>
    executor_binder<decay_t<T>, Executor>
        bind_executor(const Executor& ex, T&& t);
template<class ExecutionContext, class T>
    executor_binder<decay_t<T>, typename ExecutionContext::executor_type>
        bind_executor(ExecutionContext& ctx, T&& t);

template<class Executor>
    class executor\_work\_guard;

// make\_work\_guard:

template<class Executor>
    executor_work_guard<Executor>
        make_work_guard(const Executor& ex);
template<class ExecutionContext>
    executor_work_guard<typename ExecutionContext::executor_type>
        make_work_guard(ExecutionContext& ctx);
template<class T>
    executor_work_guard<associated_executor_t<T>>
        make_work_guard(const T& t);
template<class T, class U>
    auto make_work_guard(const T& t, U&& u)
        -> decltype(make_work_guard(get_associated_executor(t, forward<U>(u))));

class system\_executor;
class system\_context;

bool operator==(const system_executor&, const system_executor&);
bool operator!=(const system_executor&, const system_executor&);

class bad\_executor;

class executor;

bool operator==(const executor& a, const executor& b) noexcept;
bool operator==(const executor& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const executor& e) noexcept;
bool operator!=(const executor& a, const executor& b) noexcept;
bool operator!=(const executor& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const executor& e) noexcept;

// dispatch:

template<class CompletionToken>
    DEDUCED dispatch(CompletionToken&& token);
template<class Executor, class CompletionToken>
    DEDUCED dispatch(const Executor& ex, CompletionToken&& token);
template<class ExecutionContext, class CompletionToken>
    DEDUCED dispatch(ExecutionContext& ctx, CompletionToken&& token);

```

```

// post:

template<class CompletionToken>
    DEDUCED post(CompletionToken&& token);
template<class Executor, class CompletionToken>
    DEDUCED post(const Executor& ex, CompletionToken&& token);
template<class ExecutionContext, class CompletionToken>
    DEDUCED post(ExecutionContext& ctx, CompletionToken&& token);

// defer:

template<class CompletionToken>
    DEDUCED defer(CompletionToken&& token);
template<class Executor, class CompletionToken>
    DEDUCED defer(const Executor& ex, CompletionToken&& token);
template<class ExecutionContext, class CompletionToken>
    DEDUCED defer(ExecutionContext& ctx, CompletionToken&& token);

template<class Executor>
    class strand;

template<class Executor>
    bool operator==(const strand<Executor>& a, const strand<Executor>& b);
template<class Executor>
    bool operator!=(const strand<Executor>& a, const strand<Executor>& b);

template<class ProtoAllocator = allocator<void>>
    class use\_future\_t;

constexpr use_future_t<> use_future = use_future_t<>();

template<class ProtoAllocator, class Result, class... Args>
    class async_result<use_future_t<ProtoAllocator>, Result(Args...)>;

template<class R, class... Args, class Signature>
    class async\_result<packaged\_task<Result\(Args...\)>, Signature>;

} // inline namespace concurrency_v2
} // namespace experimental

template<class Allocator>
    struct uses_allocator<experimental::concurrency_v2::executor, Allocator>
        : true_type {};

} // namespace std

```

12.3. Requirements

12.3.1. Proto-allocator requirements

A type *A* meets the proto-allocator requirements if *A* is CopyConstructible (C++Std [copyconstructible]), Destructible (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where *U* is an object type. [Note: For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. —end note] No constructor, comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

12.3.2. Execution context requirements

A type *X* meets the ExecutionContext requirements if it is publicly and unambiguously derived from `execution_context`, and satisfies the additional requirements listed below.

In the table below, *x* denotes a value of type *X*.

Table 3. ExecutionContext requirements

expression	return type	assertion/note pre/post-condition

<code>X::executor_type</code>	type meeting Executor requirements	
<code>x.~X()</code>		Destroys all unexecuted function objects that were submitted via an executor object that is associated with the execution context.
<code>x.get_executor()</code>	<code>X::executor_type</code>	Returns an executor object that is associated with the execution context.

12.3.3. Executor requirements

The library describes a standard set of requirements for *executors*. A type meeting the `Executor` requirements embodies a set of rules for determining how submitted function objects are to be executed.

A type `X` meets the `Executor` requirements if it satisfies the requirements of `CopyConstructible` (C++Std [copyconstructible]) and `Destructible` (C++Std [destructible]), as well as the additional requirements listed below.

No constructor, comparison operator, copy operation, move operation, swap operation, or member functions `context`, `on_work_started`, and `on_work_finished` on these types shall exit via an exception.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

In the table below, `x1` and `x2` denote values of type `X`, `cx1` and `cx2` denote (possibly `const`) values of type `X`, `mx1` denotes an `xvalue` of type `X`, `f` denotes a `MoveConstructible` (C++Std [moveconstructible]) function object callable with zero arguments, `a` denotes a (possibly `const`) value of type `A` meeting the `Allocator` requirements (C++Std [allocator.requirements]), and `u` denotes an identifier.

Table 4. Executor requirements

expression	type	assertion/note pre/post-conditions
<code>X u(cx1);</code>		Shall not exit via an exception. post: <code>u == cx1</code> and <code>std::addressof(u.context()) == std::addressof(cx1.context())</code> .
<code>X u(mx1);</code>		Shall not exit via an exception. post: <code>u</code> equals the prior value of <code>mx1</code> and <code>std::addressof(u.context())</code> equals the prior value of <code>std::addressof(mx1.context())</code> .
<code>cx1 == cx2</code>	<code>bool</code>	Returns <code>true</code> only if <code>cx1</code> and <code>cx2</code> can be interchanged with identical effects in any of the expressions defined in these type requirements. [Note: Returning <code>false</code> does not necessarily imply that the effects are not identical. —end note] operator <code>==</code> shall be reflexive, symmetric, and transitive, and shall not exit via an exception.
<code>cx1 != cx2</code>	<code>bool</code>	Same as <code>!(cx1 == cx2)</code> .
<code>x1.context()</code>	<code>execution_context&</code> , or <code>E&</code> where <code>E</code> is a type that satisfies the ExecutionContext requirements.	Shall not exit via an exception. The comparison operators and member functions defined in these requirements shall not alter the reference returned by this function.
<code>x1.on_work_started()</code>		Shall not exit via an exception.
		Shall not exit via an exception.

x1.on_work_finished()		Precondition: A preceding call x2.on_work_started() where x1 == x2.
x1.dispatch(std::move(f),a)		Effects: Creates an object f1 initialized with <i>DECAY_COPY</i>(forward<Func>(f)) (C++Std [thread.decaycopy]) in the current thread of execution . Calls f1() at most once. The executor may block forward progress of the caller until f1() finishes execution. Executor implementations should use the supplied allocator to allocate any memory required to store the function object. Prior to invoking the function object, the executor shall deallocate any memory allocated. [Note: Executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. —end note] Synchronization: The invocation of dispatch synchronizes with (C++Std [intro.multithread]) the invocation of f1.
x1.post(std::move(f),a) x1.defer(std::move(f),a)		Effects: Creates an object f1 initialized with <i>DECAY_COPY</i>(forward<Func>(f)) in the current thread of execution. Calls f1() at most once. The executor shall not block forward progress of the caller pending completion of f1(). Executor implementations should use the supplied allocator to allocate any memory required to store the function object. Prior to invoking the function object, the executor shall deallocate any memory allocated. [Note: Executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. —end note] Synchronization: The invocation of post or defer synchronizes with (C++Std [intro.multithread]) the invocation of f1. [Note: Although the requirements placed on defer are identical to post, the use of post conveys a preference that the caller <i>does not</i> block the first step of f1's progress, whereas defer conveys a preference that the caller <i>does</i> block the first step of f1. One use of defer is to convey the intention of the caller that f1 is a continuation of the current call context. The executor may use this information to optimize or otherwise adjust the way in which f1 is invoked. —end note]

12.3.4. Service requirements

A class is a *service* if it is publicly and unambiguously derived from `execution_context::service`, or if it is publicly and unambiguously derived from another service. For a service `S`, `S::key_type` shall be valid and denote a type (C++Std [temp.deduct]), `is_base_of_v<typename S::key_type, S>` shall be true, and `S` shall satisfy the Destructible requirements (C++Std [destructible]).

The first parameter of all service constructors shall be an lvalue reference to `execution_context`. This parameter denotes the `execution_context` object that represents a set of services, of which the service object will be a member. [Note: These constructors may be called by the `make_service` function. —end note]

A service shall provide an explicit constructor with a single parameter of lvalue reference to `execution_context`. [Note: This constructor may be called by the `use_service` function. —end note]

[Example:

```
class my_service : public execution_context::service
{
```

```
public:
    typedef my_service key_type;
    explicit my_service(execution_context& ctx);
    my_service(execution_context& ctx, int some_value);
private:
    virtual void shutdown() noexcept override;
    ...
};
```

—*end example*]

A service's shutdown member function shall destroy all copies of user-defined function objects that are held by the service.

12.3.5. Signature requirements

A type satisfies the signature requirements if it is a call signature (C++Std [func.def]).

12.3.6. Associator requirements

An *associator* defines a relationship between different types and objects where, given:

- a source object *s* of type *S*,
- type requirements *R*, and
- a candidate object *c* of type *C* meeting the type requirements *R*

an *associated type* *A* meeting the type requirements *R* may be computed, and an *associated object* *a* of type *A* may be obtained.

An associator shall be a class template that takes two template type arguments. The first template argument is the source type *S*. The second template argument is the candidate type *C*. The second template argument shall be defaulted to some default candidate type *D* that satisfies the type requirements *R*.

An associator shall additionally satisfy the requirements in the table below. In this table, *X* is a class template that meets the associator requirements, *S* is the source type, *s* is a (possibly const) value of type *S*, *C* is the candidate type, *c* is a (possibly const) value of type *C*, *D* is the default candidate type, and *d* is a (possibly const) value of type *D* that is the default candidate object.

Table 5. Associator requirements

expression	return type	assertion/note pre/post-conditions
X<S>::type	X<S, D>::type	
X<S, C>::type		The associated type.
X<S>::get(s)	X<S>::type	Returns X<S>::get(S, d).
X<S, C>::get(s, c)	X<S, C>::type	Returns the associated object.

The associator's primary template shall be defined. A program may partially specialize the associator class template for some user-defined type *S*.

Finally, the associator shall provide the following type alias and function template in the enclosing namespace:

```
template<class S, class C = D> using X_t = typename X<S, C>::type;

template<class S, class C = D>
typename X<S, C>::type get_X(const S& s, const C& c = d)
{
    return X<S, C>::get(s, c);
}
```

where *x* is replaced with the name of the associator class template. [Note: This function template is provided as a

convenience, to automatically deduce the source and candidate types. —*end note*]

12.3.7. Requirements on asynchronous operations

This section uses the names `Alloc1`, `Alloc2`, `alloc1`, `alloc2`, `Args`, `CompletionHandler`, `completion_handler`, `Executor1`, `Executor2`, `ex1`, `ex2`, `f`, `i`, `N`, `Signature`, `token`, `Ti`, `ti`, `work1`, and `work2` as placeholders for specifying the requirements below.

12.3.7.1. General asynchronous operation concepts

An *initiating function* is a function which may be called to start an asynchronous operation. A *completion handler* is a function object that will be invoked, at most once, with the result of the asynchronous operation.

The lifecycle of an asynchronous operation is comprised of the following events and phases:

- Event 1: The asynchronous operation is started by a call to the *initiating function*.
- Phase 1: The asynchronous operation is now *outstanding*.
- Event 2: The externally observable side effects of the asynchronous operation, if any, are fully established. The *completion handler* is submitted to an executor.
- Phase 2: The asynchronous operation is now *completed*.
- Event 3: The completion handler is called with the result of the asynchronous operation.

In this Technical Specification, all functions with the prefix `async_` are initiating functions.

12.3.7.2. Completion tokens and handlers

Initiating functions:

- are function templates with template parameter `CompletionToken`;
- accept, as the final parameter, a *completion token* object `token` of type `CompletionToken`;
- specify a *completion signature*, which is a call signature (C++Std [func.def]) `Signature` that determines the arguments to the completion handler.

An initiating function determines the type `CompletionHandler` of its completion handler function object by performing `typename async_result<decay_t<CompletionToken>, Signature>::completion_handler_type`. The completion handler object `completion_handler` is initialized with `forward<CompletionToken>(token)`. [*Note*: No other requirements are placed on the type `CompletionToken`. —*end note*]

The type `CompletionHandler` must satisfy the requirements of `Destructible` (C++Std [destructible]) and `MoveConstructible` (C++Std [moveconstructible]), and be callable with the specified call signature.

In this Technical Specification, all initiating functions specify a *Completion signature* element that defines the call signature `Signature`. The *Completion signature* elements in this Technical Specification have named parameters, and the results of an asynchronous operation are specified in terms of these names.

12.3.7.3. Automatic deduction of initiating function return type

The return type of an initiating function is `typename async_result<decay_t<CompletionToken>, Signature>::return_type`.

For the sake of exposition, this Technical Specification sometimes annotates functions with a return type *DEDUCED*. For every function declaration that returns *DEDUCED*, the meaning is equivalent to specifying the return type as `typename async_result<decay_t<CompletionToken>, Signature>::return_type`.

12.3.7.4. Production of initiating function return value

An initiating function produces its return type as follows:

- constructing an object `result` of type `async_result<decay_t<CompletionToken>, Signature>`, initialized as `result(completion_handler)`; and
- using `result.get()` as the operand of the return statement.

[Example: Given an asynchronous operation with *Completion signature* `void(R1 r1, R2 r2)`, an initiating function meeting these requirements may be implemented as follows:

```
template<class CompletionToken>
auto async_xyz(T1 t1, T2 t2, CompletionToken&& token)
{
    typename async_result<decay_t<CompletionToken>, void(R1, R2)>::completion_handler_type
        completion_handler(forward<CompletionToken>(token));

    async_result<decay_t<CompletionToken>, void(R1, R2)> result(completion_handler);

    // initiate the operation and cause completion_handler to be invoked with
    // the result

    return result.get();
}
```

For convenience, initiating functions may be implemented using the `async_completion` template:

```
template<class CompletionToken>
auto async_xyz(T1 t1, T2 t2, CompletionToken&& token)
{
    async_completion<CompletionToken, void(R1, R2)> init(token);

    // initiate the operation and cause init.completion_handler to be invoked
    // with the result

    return init.result.get();
}
```

—end example]

12.3.7.5. Lifetime of initiating function arguments

Unless otherwise specified, the lifetime of arguments to initiating functions shall be treated as follows:

— If the parameter has a pointer type or has a type of lvalue reference to non-const, the implementation may assume the validity of the pointee or referent, respectively, until the completion handler is invoked. [Note: In other words, the program must guarantee the validity of the argument until the completion handler is invoked. —end note]

— Otherwise, the implementation must not assume the validity of the argument after the initiating function completes. [Note: In other words, the program is not required to guarantee the validity of the argument after the initiating function completes. —end note] The implementation may make copies of the argument, and all copies shall be destroyed no later than immediately after invocation of the completion handler.

12.3.7.6. Non-blocking requirements on initiating functions

An initiating function shall not block (C++Std [defs.block]) the calling thread pending completion of the outstanding operation.

[Note: Initiating functions may still block the calling thread for other reasons. For example, an initiating function may lock a mutex in order to synchronize access to shared data. —end note]

12.3.7.7. Associated executor

Certain objects that participate in asynchronous operations have an *associated executor*. These are obtained as specified below.

12.3.7.8. I/O executor

An asynchronous operation has an associated executor satisfying the [Executor](#) requirements. If not otherwise specified by the asynchronous operation, this associated executor is an object of type `system_executor`.

All asynchronous operations in this Technical Specification have an associated executor object that is determined as follows:

— If the initiating function is a member function, the associated executor is that returned by the `get_executor` member function on the same object.

— If the initiating function is not a member function, the associated executor is that returned by the `get_executor` member function of the first argument to the initiating function.

Let `Executor1` be the type of the associated executor. Let `ex1` be a value of type `Executor1`, representing the associated executor object obtained as described above.

12.3.7.9. Completion handler executor

A completion handler object of type `CompletionHandler` has an associated executor of type `Executor2` satisfying the [Executor requirements](#). The type `Executor2` is `associated_executor_t<CompletionHandler, Executor1>`. Let `ex2` be a value of type `Executor2` obtained by performing `get_associated_executor(completion_handler, ex1)`.

12.3.7.10. Outstanding work

The implementation of an asynchronous operation shall maintain an object `work1` of type `executor_work_guard<Executor1>`, initialized with `work1(ex1)` and with `work1.owns_work() == true`, until the asynchronous operation has completed.

The implementation of an asynchronous operation shall maintain an object `work2` of type `executor_work_guard<Executor2>`, initialized with `work2(ex2)` and with `work2.owns_work() == true`, until the asynchronous operation has completed and `completion_handler` has been submitted for execution.

12.3.7.11. Allocation of intermediate storage

Asynchronous operations may allocate memory. [*Note:* Such as a data structure to store copies of the `completion_handler` object and the initiating function's arguments. —*end note*]

Let `Alloc1` be a type, satisfying the [ProtoAllocator](#) requirements, that represents the asynchronous operation's default allocation strategy. [*Note:* Typically `std::allocator<void>`. —*end note*] Let `alloc1` be a value of type `Alloc1`.

A completion handler object of type `CompletionHandler` has an associated allocator object `alloc2` of type `Alloc2` satisfying the [ProtoAllocator](#) requirements. The type `Alloc2` is `associated_allocator_t<CompletionHandler, Alloc1>`. Let `alloc2` be a value of type `Alloc2` obtained by performing `get_associated_allocator(completion_handler, alloc1)`.

The asynchronous operations defined in this Technical Specification:

— If required, allocate memory using only the completion handler's associated allocator.

— Prior to completion handler execution, deallocate any memory allocated using the completion handler's associated allocator.

[*Note:* The implementation may perform operating system or underlying API calls that perform memory allocations not using the associated allocator. Invocations of the allocator functions may not introduce data races (See C++Std [res.on.data.races]). —*end note*]

12.3.7.12. Execution of completion handler on completion of asynchronous operation

Let `Args...` be the argument types of the completion signature `Signature` and let N be `sizeof...(Args)`. Let i be in the range $[0, N)$. Let τ_i be the i th type in `Args...` and let t_i be the i th completion handler argument associated with τ_i .

Let `f` be a function object, callable as `f()`, that invokes `completion_handler` as if by `completion_handler(forward<T0>(t0), ..., forward<TN-1>(tN-1))`.

If an asynchronous operation completes immediately (that is, within the thread of execution calling the initiating function, and before the initiating function returns), the completion handler shall be submitted for execution as if by performing `ex2.post(std::move(f), alloc2)`. Otherwise, the completion handler shall be submitted for execution as if by performing `ex2.dispatch(std::move(f), alloc2)`.

12.3.7.13. Completion handlers and exceptions

Completion handlers are permitted to throw exceptions. The effect of any exception propagated from the execution of a completion handler is determined by the executor which is executing the completion handler.

12.4. Class template `async_result`

The `async_result` class template is a customization point for asynchronous operations. Template parameter

CompletionToken specifies the model used to obtain the result of the asynchronous operation. Template parameter Signature is the call signature (C++Std [func.def]) for the completion handler type invoked on completion of the asynchronous operation. The async_result template:

- transforms a CompletionToken into a completion handler type that is based on a Signature; and
- determines the return type and return value of an asynchronous operation's initiating function.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class CompletionToken, class Signature, class = void>
    class async_result
    {
    public:
        typedef CompletionToken completion_handler_type;
        typedef void return_type;

        explicit async_result(completion_handler_type&) {}
        async_result(const async_result&) = delete;
        async_result& operator=(const async_result&) = delete;

        return_type get() {}
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

The template parameter CompletionToken shall be an object type. The template parameter Signature shall be a call signature (C++Std [func.def]).

Specializations of async_result shall satisfy the Destructible requirements (C++Std [destructible]) in addition to the requirements in the table below. In this table, R is a specialization of async_result; r is a modifiable lvalue of type R; and h is a modifiable lvalue of type R::completion_handler_type.

Table 6. async_result specialization requirements

Expression	Return type	Requirement
R::completion_handler_type		A type satisfying MoveConstructible requirements (C++Std [moveconstructible]), An object of type completion_handler_type shall be a function object with call signature Signature, and completion_handler_type shall be constructible with an rvalue of type CompletionToken.
R::return_type		void; or a type satisfying MoveConstructible requirements (C++Std [moveconstructible])
R r(h);		
r.get()	R::return_type	[Note: An asynchronous operation's initiating function uses the get() member function as the sole operand of a return statement. —end note]

12.5. Class template async_completion

Class template async_completion is provided as a convenience, to simplify the implementation of asynchronous operations that use async_result.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class CompletionToken, class Signature>
    struct async_completion
```

```

{
    typedef async_result<decay_t<CompletionToken>,
        Signature>::completion_handler_type
        completion_handler_type;

    explicit async_completion(CompletionToken& t);
    async_completion(const async_completion&) = delete;
    async_completion& operator=(const async_completion&) = delete;

    see below completion_handler;
    async_result<decay_t<CompletionToken>, Signature> result;
};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The template parameter `Signature` shall be a call signature (C++Std [func.def]).

```
explicit async_completion(CompletionToken& t);
```

Effects: If `CompletionToken` and `completion_handler_type` are the same type, binds `completion_handler` to `t`; otherwise, initializes `completion_handler` with the result of `forward<CompletionToken>(t)`. Initializes `result` with `completion_handler`.

see below completion_handler;

Type: `completion_handler_type&` if `CompletionToken` and `completion_handler_type` are the same type; otherwise, `completion_handler_type`.

12.6. Class template associated_allocator

Class template `associated_allocator` is an [associator](#) for the [ProtoAllocator](#) type requirements, with default candidate type `allocator<void>` and default candidate object `allocator<void>()`.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T, class ProtoAllocator = allocator<void>>
    struct associated_allocator
    {
        typedef see below type;

        static type get(const T& t, const ProtoAllocator& a = ProtoAllocator()) noexcept;
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

Specializations of `associated_allocator` shall satisfy the requirements in the table below. In this table, `X` is a specialization of `associated_allocator` for the template parameters `T` and `ProtoAllocator`; `t` is a value of (possibly `const`) `T`; and `a` is an object of type `ProtoAllocator`.

Table 7. associated_allocator specialization requirements

Expression	Return type	Note
typename <code>X::type</code>	A type meeting the proto-allocator requirements.	
<code>X::get(t)</code>	<code>X::type</code>	Shall not exit via an exception. Equivalent to <code>X::get(t, ProtoAllocator())</code> .
<code>X::get(t, a)</code>	<code>X::type</code>	Shall not exit via an exception.

12.6.1. associated_allocator members

typedef *see below* type;

Type: If T has a nested type `allocator_type`, `typename T::allocator_type`. Otherwise `ProtoAllocator`.

`type get(const T& t, const ProtoAllocator& a = ProtoAllocator()) noexcept;`

Returns: If T has a nested type `allocator_type`, `t.get_allocator()`. Otherwise `a`.

12.7. Function `get_associated_allocator`

```
template<class T>
associated_allocator_t<T> get_associated_allocator(const T& t) noexcept;
```

Returns: `associated_allocator<T>::get(t)`.

```
template<class T, class ProtoAllocator>
associated_allocator_t<T, ProtoAllocator>
get_associated_allocator(const T& t, const ProtoAllocator& a) noexcept;
```

Returns: `associated_allocator<T, ProtoAllocator>::get(t, a)`.

12.8. Class `execution_context`

Class `execution_context` implements an extensible, type-safe, polymorphic set of services, indexed by service type.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

class execution_context
{
public:
class service;

// construct / copy / destroy:

execution_context();
execution_context(const execution_context&) = delete;
execution_context& operator=(const execution_context&) = delete;
virtual ~execution_context();

// execution context operations:

void notify_fork(fork_event e);

protected:

// execution context protected operations:

void shutdown() noexcept;
void destroy() noexcept;
};

// service access:
template<class Service> typename Service::key_type&
use_service(execution_context& ctx);
template<class Service, class... Args> Service&
make_service(execution_context& ctx, Args&&... args);
template<class Service> bool has_service(const execution_context& ctx) noexcept;
class service_already_exists : public logic_error { };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

Access to the services of an `execution_context` is via three function templates, `use_service<>`, `make_service<>` and `has_service<>`.

In a call to `use_service<Service>()`, the type argument chooses a service. If the service is not present in an `execution_context`, an object of type `Service` is created and added to the `execution_context`. A program can check if an `execution_context` implements a particular service with the function template `has_service<Service>()`.

Service objects may be explicitly added to an `execution_context` using the function template `make_service<Service>()`. If the service is already present, `make_service` exits via an exception of type `service_already_exists`.

Once a service reference is obtained from an `execution_context` object by calling `use_service<>`, that reference remains usable until a call to `destroy()`.

12.8.1. `execution_context` constructor

```
execution_context();
```

Effects: Creates an object of class `execution_context` which contains no services. [Note: An implementation might preload services of internal service types for its own use. —end note]

12.8.2. `execution_context` destructor

```
~execution_context();
```

Effects: Destroys an object of class `execution_context`. Performs `shutdown()` followed by `destroy()`.

12.8.3. `execution_context` operations

```
void notify_fork(fork_event e);
```

Effects: For each service object `svc` in the set:

- If `e == fork_event::prepare`, performs `svc->notify_fork(e)` in reverse order of addition to the set.
- Otherwise, performs `svc->notify_fork(e)` in order of addition to the set.

12.8.4. `execution_context` protected operations

```
void shutdown() noexcept;
```

Effects: For each service object `svc` in the `execution_context` set, in reverse order of addition to the set, performs `svc->shutdown()`. For each service in the set, `svc->shutdown()` is called only once irrespective of the number of calls to `shutdown` on the `execution_context`.

```
void destroy() noexcept;
```

Effects: Destroys each service object in the `execution_context` set, and removes it from the set, in reverse order of addition to the set.

12.8.5. `execution_context` globals

The functions `use_service`, `make_service`, and `has_service` do not introduce data races as a result of concurrent calls to those functions from different threads.

```
template<class Service> typename Service::key_type&  
use_service(execution_context& ctx);
```

Effects: If an object of type `Service::key_type` does not already exist in the `execution_context` set identified by `ctx`, creates an object of type `Service`, initialized as `Service(ctx)`, and adds it to the set.

Returns: A reference to the corresponding service of `ctx`.

Notes: The reference returned remains valid until a call to `destroy`.

```
template<class Service, class... Args> Service&  
make_service(execution_context& ctx, Args&&... args);
```

Requires: A service object of type `Service::key_type` does not already exist in the `execution_context` set identified by `ctx`.

Effects: Creates an object of type `Service`, initialized as `Service(ctx, forward<Args>(args)...)` , and adds it to the `execution_context` set identified by `ctx`.

Throws: `service_already_exists` if a corresponding service object of type `Key` is already present in the set.

Notes: The reference returned remains valid until a call to `destroy`.

```
template<class Service> bool has_service(const execution_context& ctx) noexcept;
```

Returns: true if an object of type `Service::key_type` is present in `ctx`, otherwise false.

12.9. Class `execution_context::service`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class execution_context::service
    {
    protected:
        // construct / copy / destroy:

        explicit service(execution_context& owner);
        service(const service&) = delete;
        service& operator=(const service&) = delete;
        virtual ~service();

        // service observers:

        execution_context& context() noexcept;

    private:
        // service operations:

        virtual void shutdown() noexcept = 0;
        virtual void notify_fork(fork_event e) {}

        execution_context& context_; // exposition only
    };
} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

```
explicit service(execution_context& owner);
```

Postconditions: `std::addressof(context_) == std::addressof(owner)`.

```
execution_context& context() noexcept;
```

Returns: `context_`.

12.10. Class template `is_executor`

The class template `is_executor` can be used to detect executor types satisfying the [Executor](#) type requirements.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T> struct is_executor;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

`T` shall be a complete type.

Class template `is_executor` is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the type `T` meets the syntactic requirements for [Executor](#), otherwise `false_type`.

12.11. Executor argument tag

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    struct executor_arg_t { };
    constexpr executor_arg_t executor_arg = executor_arg_t();

}
```

```

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The `executor_arg_t` struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, types may have constructors with `executor_arg_t` as the first argument, immediately followed by an argument of a type that satisfies the [Executor requirements](#).

12.12. uses_executor

12.12.1. uses_executor trait

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T, class Executor> struct uses_executor;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

Remark: Detects whether `T` has a nested `executor_type` that is convertible from `Executor`. Meets the `BinaryTypeTrait` requirements (C++Std [meta.rqmts]). The implementation provides a definition that is derived from `true_type` if a type `T::executor_type` exists and `is_convertible<Executor, T::executor_type>::value != false`, otherwise it is derived from `false_type`. A program may specialize this template to derive from `true_type` for a user-defined type `T` that does not have a nested `executor_type` but nonetheless can be constructed with an executor if the first argument of a constructor has type `executor_arg_t` and the second argument has type `Executor`.

12.12.2. uses-executor construction

Uses-executor construction with executor `Executor` refers to the construction of an object `obj` of type `T`, using constructor arguments `v1`, `v2`, ..., `vN` of types `V1`, `V2`, ..., `VN`, respectively, and an executor `ex` of type `Executor`, according to the following rules:

— if `uses_executor<T, Executor>::value` is true and `is_constructible<T, executor_arg_t, Executor, V1, V2, ..., VN>::value` is true, then `obj` is initialized as `obj(executor_arg, ex, v1, v2, ..., vN)`;

— otherwise, `obj` is initialized as `obj(v1, v2, ..., vN)`.

12.13. Class template associated_executor

Class template `associated_allocator` is an [associator](#) for the [Executor](#) type requirements, with default candidate type `system_executor` and default candidate object `system_executor()`.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T, class Executor = system_executor>
    struct associated_executor
    {
        typedef see below type;

        static type get(const T& t, const Executor& e = Executor()) noexcept;
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

Specializations of `associated_executor` shall satisfy the requirements in the table below. In this table, `x` is a specialization of `associated_executor` for the template parameters `T` and `Executor`; `t` is a value of (possibly const) `T`; and `e` is an object of type `Executor`.

Table 8. associated_executor specialization requirements

--	--	--

Expression	Return type	Note
typename X::type	A type meeting Executor requirements .	
X::get(t)	X::type	Shall not exit via an exception. Equivalent to X::get(t, Executor()).
X::get(t, e)	X::type	Shall not exit via an exception.

12.13.1. associated_executor members

typedef *see below* type;

Type: If T has a nested type executor_type, typename T::executor_type. Otherwise Executor.

type get(const T& t, const Executor& e = Executor()) noexcept;

Returns: If T has a nested type executor_type, t.get_executor(). Otherwise e.

12.14. Function get_associated_executor

```
template<class T>
associated_executor_t<T> get_associated_executor(const T& t) noexcept;
```

Returns: associated_executor<T>::get(t).

```
template<class T, class Executor>
associated_executor_t<T, Executor>
get_associated_executor(const T& t, const Executor& ex) noexcept;
```

Returns: associated_executor<T, Executor>::get(t, ex).

Remarks: This function shall not participate in overload resolution unless is_executor<Executor>::value is true.

```
template<class T, class ExecutionContext>
associated_executor_t<T, typename ExecutionContext::executor_type>
get_associated_executor(const T& t, ExecutionContext& ctx) noexcept;
```

Returns: get_associated_executor(t, ctx.get_executor()).

Remarks: This function shall not participate in overload resolution unless is_convertible<ExecutionContext&, execution_context&>::value is true.

12.15. Class template executor_binder

executor_binder<T, Executor> binds an executor of type Executor satisfying [Executor requirements](#) to an object or function of type T.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

template<class T, class Executor>
class executor_binder
{
public:
// types:

typedef T target_type;
typedef Executor executor_type;

// construct / copy / destroy:

executor_binder(T t, const Executor& ex);
executor_binder(const executor_binder& other) = default;
executor_binder(executor_binder&& other) = default;
template<class U, class OtherExecutor>
executor_binder(const executor_binder<U, OtherExecutor>& other);
```

```

template<class U, class OtherExecutor>
    executor_binder(executor_binder<U, OtherExecutor>&& other);
template<class U, class OtherExecutor>
    executor_binder(executor_arg_t, const Executor& ex,
        const executor_binder<U, OtherExecutor>& other);
template<class U, class OtherExecutor>
    executor_binder(executor_arg_t, const Executor& ex,
        executor_binder<U, OtherExecutor>&& other);

~executor_binder();

// executor binder access:

T& get() noexcept;
const T& get() const noexcept;
executor_type get_executor() const noexcept;

// executor binder invocation:

template<class... Args>
    result_of_t<T&(Args&&...)> operator()(Args&&... args);
template<class... Args>
    result_of_t<const T&(Args&&...)> operator()(Args&&... args) const;

private:
    Executor ex_; // exposition only
    T target_; // exposition only
};

template<class T, class Executor, class Signature>
    class async_result<executor_binder<T, Executor>, Signature>;

template<class T, class Executor, class ProtoAllocator>
    struct associated_allocator<executor_binder<T, Executor>, ProtoAllocator>;

template<class T, class Executor, class Executor1>
    struct associated_executor<executor_binder<T, Executor>, Executor1>;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

12.15.1. executor_binder constructors

```
executor_binder(T t, const Executor& ex);
```

Effects: Initializes `ex_` with `ex`. Initializes `target_` by performing uses-executor construction, using the constructor argument `std::move(t)` and the executor `ex_`.

```
template<class U, class OtherExecutor>
    executor_binder(const executor_binder<U, OtherExecutor>& other);
```

Requires: If `U` is not convertible to `T`, or if `OtherExecutor` is not convertible to `Executor`, the program is ill-formed.

Effects: Initializes `ex_` with `other.get_executor()`. Initializes `target_` by performing uses-executor construction, using the constructor argument `other.get()` and the executor `ex_`.

```
template<class U, class OtherExecutor>
    executor_binder(executor_binder<U, OtherExecutor>&& other);
```

Requires: If `U` is not convertible to `T`, or if `OtherExecutor` is not convertible to `Executor`, the program is ill-formed.

Effects: Initializes `ex_` with `other.get_executor()`. Initializes `target_` by performing uses-executor construction, using the constructor argument `std::move(other.get())` and the executor `ex_`.

```
template<class U, class OtherExecutor>
    executor_binder(executor_arg_t, const Executor& ex,
        const executor_binder<U, OtherExecutor>& other);
```

Requires: If `U` is not convertible to `T` the program is ill-formed.

Effects: Initializes `ex_` with `ex`. Initializes `target_` by performing uses-executor construction, using the constructor argument `other.get()` and the executor `ex_`.

```
template<class U, class OtherExecutor>
    executor_binder(executor_arg_t, const Executor& ex,
        executor_binder<U, OtherExecutor>&& other);
```

Requires: `U` is `T` or convertible to `T`.

Effects: Initializes `ex_` with `ex`. Initializes `target_` by performing uses-executor construction, using the constructor argument `std::move(other.get())` and the executor `ex_`.

12.15.2. executor_binder access

```
T& get() noexcept;
const T& get() const noexcept;
```

Returns: `target_`.

```
executor_type get_executor() const noexcept;
```

Returns: `executor_`.

12.15.3. executor_binder invocation

```
template<class... Args>
    result_of_t<T&(Args&&...)> operator()(Args&&... args);
template<class... Args>
    result_of_t<const T&(Args&&...)> operator()(Args&&... args) const;
```

Returns: *INVOKE*(`get()`, `forward<Args>(args)...`) (C++Std [func.require]).

12.15.4. Class template partial specialization `async_result`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T, class Executor, class Signature>
    class async_result<executor_binder<T, Executor>, Signature>
    {
    public:
        typedef executor_binder<
            typename async_result<T, Signature>::completion_handler_type,
            Executor> completion_handler_type;
        typedef typename async_result<T, Signature>::return_type return_type;

        explicit async_result(completion_handler_type& h);
        async_result(const async_result&) = delete;
        async_result& operator=(const async_result&) = delete;

        return_type get();

    private:
        async_result<T, Signature> target_; // exposition only
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

```
explicit async_result(completion_handler_type& h);
```

Effects: Initializes `target_` as `target_(h.get())`.

```
return_type get();
```

Returns: `target_.get()`.

12.15.5. Class template partial specialization `associated_allocator`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T, class Executor, class ProtoAllocator>
        struct associated_allocator<executor_binder<T, Executor>, ProtoAllocator>
        {
            typedef associated_allocator_t<T, ProtoAllocator> type;

            static type get(const executor_binder<T, Executor>& b,
                           const ProtoAllocator& a = ProtoAllocator()) noexcept;
        };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

```
static type get(const executor_binder<T, Executor>& b,
               const ProtoAllocator& a = ProtoAllocator()) noexcept;
```

Returns: `associated_allocator<T, ProtoAllocator>::get(b.get(), a)`.

12.15.6. Class template partial specialization `associated_executor`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T, class Executor, class Executor1>
        struct associated_executor<executor_binder<T, Executor>, Executor1>
        {
            typedef Executor type;

            static type get(const executor_binder<T, Executor>& b,
                           const Executor1& e = Executor1()) noexcept;
        };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

```
static type get(const executor_binder<T, Executor>& b,
               const Executor1& e = Executor1()) noexcept;
```

Returns: `b.get_executor()`.

12.16. Function `bind_executor`

```
template<class Executor, class T>
    executor_binder<decay_t<T>, Executor>
    bind_executor(const Executor& ex, T&& t);
```

Returns: `executor_binder<decay_t<T>, Executor>(forward<T>(t), ex)`.

Remarks: This function shall not participate in overload resolution unless `is_executor<Executor>::value` is true.

```
template<class ExecutionContext, class CompletionToken>
    executor_binder<decay_t<T>, typename ExecutionContext::executor_type>
    bind_executor(ExecutionContext& ctx, T&& t);
```

Returns: `bind_executor(ctx.get_executor(), forward<T>(t))`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t>::value` is true.

12.17. Class template `executor_work_guard`

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class Executor>
    class executor_work_guard
    {
    public:
        // types:

        typedef Executor executor_type;

        // construct / copy / destroy:

        explicit executor_work_guard(const executor_type& ex) noexcept;
        executor_work_guard(const executor_work_guard& other) noexcept;
        executor_work_guard(executor_work_guard&& other) noexcept;

        executor_work_guard& operator=(const executor_work_guard&) = delete;

        ~executor_work_guard();

        // executor work guard observers:

        executor_type get_executor() const noexcept;
        bool owns_work() const noexcept;

        // executor work guard modifiers:

        void reset() noexcept;

    private:
        Executor ex_; // exposition only
        bool owns_; // exposition only
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

12.17.1. executor_work_guard members

```
explicit executor_work_guard(const executor_type& ex) noexcept;
```

Effects: Initializes `ex_` with `ex`, and then performs `ex_.on_work_started()`.

Postconditions: `ex == ex_` and `owns_ == true`.

```
executor_work_guard(const executor_work_guard& other) noexcept;
```

Effects: Initializes `ex_` with `other.ex_`. If `other.owns_ == true`, performs `ex_.on_work_started()`.

Postconditions: `ex_ == other.ex_` and `owns_ == other.owns_`.

```
executor_work_guard(executor_work_guard&& other) noexcept;
```

Effects: Initializes `ex_` with `std::move(other.ex_)` and `owns_` with `other.owns_`, and sets `other.owns_` to false.

```
~executor_work_guard();
```

Effects: If `owns_` is true, performs `ex_.on_work_finished()`.

```
executor_type get_executor() const noexcept;
```

Returns: `ex_`.

```
bool owns_work() const noexcept;
```

Returns: `owns_`.

```
void reset() noexcept;
```

Effects: If `owns_` is true, performs `ex_.on_work_finished()`.

Postconditions: `owns_ == false`.

12.18. Function `make_work_guard`

```
template<class Executor>
    executor_work_guard<Executor>
        make_work_guard(const Executor& ex);
```

Returns: `executor_work_guard<Executor>(ex)`.

Remarks: This function shall not participate in overload resolution unless `is_executor<Executor>::value` is true.

```
template<class ExecutionContext>
    executor_work_guard<typename ExecutionContext::executor_type>
        make_work_guard(ExecutionContext& ctx);
```

Returns: `make_work_guard(ctx.get_executor())`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>` is true.

```
template<class T>
    executor_work_guard<associated_executor_t<T>>
        make_work_guard(const T& t);
```

Returns: `make_work_guard(get_associated_executor(t))`.

Remarks: This function shall not participate in overload resolution unless `is_executor<T>::value` is false and `is_convertible<T&, execution_context_t::value>` is false.

```
template<class T, class U>
    auto make_work_guard(const T& t, U&& u)
        -> decltype(make_work_guard(get_associated_executor(t, forward<U>(u))));
```

Returns: `make_work_guard(get_associated_executor(t, forward<U>(u)))`.

12.19. Class `system_executor`

Class `system_executor` represents a set of rules where function objects are permitted to execute on any thread.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class system_executor
    {
    public:
        // constructors:

        system_executor() {}

        // executor operations:

        system_context& context() noexcept;

        void on_work_started() noexcept {}
        void on_work_finished() noexcept {}

        template<class Func, class ProtoAllocator>
            void dispatch(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void post(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void defer(Func&& f, const ProtoAllocator& a);
    };

    bool operator==(const system_executor&, const system_executor&) noexcept;
    bool operator!=(const system_executor&, const system_executor&) noexcept;
```

```

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

Class `system_executor` satisfies the `Destructible` (C++Std [destructible]), `DefaultConstructible` (C++Std [defaultconstructible]), and [Executor](#) type requirements.

To satisfy the `Executor` requirements for the `post` and `defer` member functions, the system executor may create thread objects to run the submitted function objects. These thread objects are collectively referred to as *system threads*.

12.19.1. `system_executor` operations

```
system_context& context() noexcept;
```

Returns: A reference to an object with static storage duration. All calls to this function return references to the same object.

```
template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a);
```

Effects: Equivalent to `DECAY_COPY(forward<Func>(f))()` (C++Std [thread.decaycopy]).

```
template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a);
template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a);
```

Effects: If `context().stopped() == false`, creates an object `f1` initialized with `DECAY_COPY(forward<Func>(f))`, and calls `f1` as if in a thread of execution represented by a thread object. Any exception propagated from the execution of `DECAY_COPY(forward<Func>(f))()` results in a call to `std::terminate`.

12.19.2. `system_executor` comparisons

```
bool operator==(const system_executor&, const system_executor&) noexcept;
```

Returns: `true`.

```
bool operator!=(const system_executor&, const system_executor&) noexcept;
```

Returns: `false`.

12.20. Class `system_context`

Class `system_context` implements the execution context associated with `system_executor` objects.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class system_context : public execution_context
    {
    public:
        // types:

        typedef system\_executor executor_type;

        // construct / copy / destroy:

        system_context() = delete;
        system_context(const system_context&) = delete;
        system_context& operator=(const system_context&) = delete;
        ~system_context();

        // system_context operations:

        executor_type get_executor() noexcept;

        void stop();
        bool stopped() const noexcept;
    };
}
}
}

```

```

    void join();
};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The class `system_context` satisfies the [ExecutionContext](#) type requirements.

The `system_context` member functions `get_executor`, `stop`, and `stopped`, and the `system_executor` copy constructors, member functions and comparison operators, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

```
~system_context();
```

Effects: Performs `stop()` followed by `join()`.

```
executor_type get_executor() noexcept;
```

Returns: `system_executor()`.

```
void stop();
```

Effects: Signals all system threads to exit as soon as possible. If a system thread is currently executing a function object, the thread will exit only after completion of that function object. Returns without waiting for the system threads to complete.

Postconditions: `stopped() == true`.

```
bool stopped() const noexcept;
```

Returns: `true` if the `system_context` has been stopped by a prior call to `stop`.

```
void join();
```

Effects: Blocks the calling thread (C++Std [defns.block]) until all system threads have completed.

Synchronization: The completion of each system thread synchronizes with (C++Std [intro.multithread]) the corresponding successful `join()` return.

12.21. Class `bad_executor`

An exception of type `bad_executor` is thrown by `executor` member functions `dispatch`, `post`, and `defer` when the `executor` object has no target.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class bad_executor : public exception
    {
    public:
        // constructor:
        bad_executor() noexcept;
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

```
bad_executor() noexcept;
```

Effects: constructs a `bad_executor` object.

Postconditions: `what()` returns an implementation-defined NTBS.

12.22. Class `executor`

The `executor` class provides a polymorphic wrapper for types that satisfy the [Executor requirements](#).

```
namespace std {
```

```

namespace experimental {
inline namespace concurrency_v2 {

class executor
{
public:
    // construct / copy / destroy:

    executor() noexcept;
    executor(nullptr_t) noexcept;
    executor(const executor& e) noexcept;
    executor(executor&& e) noexcept;
    template<class Executor> executor(Executor e);
    template<class Executor, class ProtoAllocator>
        executor(allocator_arg_t, const ProtoAllocator& a, Executor e);

    executor& operator=(const executor& e) noexcept;
    executor& operator=(executor&& e) noexcept;
    executor& operator=(nullptr_t) noexcept;
    template<class Executor> executor& operator=(Executor e);

    ~executor();

    // executor modifiers:

    void swap(executor& other) noexcept;
    template<class Executor, class ProtoAllocator>
        void assign(Executor e, const ProtoAllocator& a);

    // executor operations:

    execution_context& context() noexcept;

    void on_work_started() noexcept;
    void on_work_finished() noexcept;

    template<class Func, class ProtoAllocator>
        void dispatch(Func&& f, const ProtoAllocator& a);
    template<class Func, class ProtoAllocator>
        void post(Func&& f, const ProtoAllocator& a);
    template<class Func, class ProtoAllocator>
        void defer(Func&& f, const ProtoAllocator& a);

    // executor capacity:

    explicit operator bool() const noexcept;

    // executor target access:

    const type_info& target_type() const noexcept;
    template<class Executor> Executor* target() noexcept;
    template<class Executor> const Executor* target() const noexcept;
};

template<> struct is_executor<executor> : true_type {};

// executor comparisons:

bool operator==(const executor& a, const executor& b) noexcept;
bool operator==(const executor& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const executor& e) noexcept;
bool operator!=(const executor& a, const executor& b) noexcept;
bool operator!=(const executor& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const executor& e) noexcept;

// executor specialized algorithms:

void swap(executor& a, executor& b) noexcept;

} // inline namespace concurrency_v2
} // namespace experimental

```

```

template<class Allocator>
struct uses_allocator<experimental::concurrency_v2::executor, Allocator>
: true_type {};

} // namespace std

```

Class `executor` meets the requirements of [Executor](#), `DefaultConstructible` (C++Std [defaultconstructible]), and `CopyAssignable` (C++Std [copyassignable]).

[*Note:* To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more executor objects. —*end note*]

The *target* is the executor object that is held by the wrapper.

12.22.1. executor constructors

```
executor() noexcept;
```

Postconditions: `!*this`.

```
executor(nullptr_t) noexcept;
```

Postconditions: `!*this`.

```
executor(const executor& e) noexcept;
```

Postconditions: `!*this` if `!e`; otherwise, `*this` targets `e.target()` or a copy of `e.target()`.

```
executor(executor&& e) noexcept;
```

Effects: If `!e`, `*this` has no target; otherwise, moves `e.target()` or move-constructs the target of `e` into the target of `*this`, leaving `e` in a valid state with an unspecified value.

```
template<class Executor> executor(Executor e);
```

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

```
template<class Executor, class ProtoAllocator>
executor(allocator_arg_t, const ProtoAllocator& a, Executor e);
```

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

A copy of the allocator argument is used to allocate memory, if necessary, for the internal data structures of the constructed executor object.

12.22.2. executor assignment

```
executor& operator=(const executor& e) noexcept;
```

Effects: `executor(e).swap(*this)`.

Returns: `*this`.

```
executor& operator=(executor&& e) noexcept;
```

Effects: Replaces the target of `*this` with the target of `e`, leaving `e` in a valid state with an unspecified value.

Returns: `*this`.

```
executor& operator=(nullptr_t) noexcept;
```

Effects: `executor(nullptr).swap(*this)`.

Returns: `*this`.

```
template<class Executor> executor& operator=(Executor e);
```

Effects: `executor(std::move(e)).swap(*this)`.

Returns: `*this`.

12.22.3. executor destructor

```
~executor();
```

Effects: If `*this != nullptr`, releases shared ownership of, or destroys, the target of `*this`.

12.22.4. executor modifiers

```
void swap(executor& other) noexcept;
```

Effects: Interchanges the targets of `*this` and `other`.

```
template<class Executor, class ProtoAllocator>
void assign(Executor e, const ProtoAllocator& a);
```

Effects: `executor(allocator_arg, a, std::move(e)).swap(*this)`.

12.22.5. executor operations

```
execution_context& context() noexcept;
```

Requires: `*this != nullptr`.

Returns: `e.context()`, where `e` is the target object of `*this`.

```
void on_work_started() noexcept;
```

Requires: `*this != nullptr`.

Effects: `e.on_work_started()`, where `e` is the target object of `*this`.

```
void on_work_finished() noexcept;
```

Requires: `*this != nullptr`.

Effects: `e.on_work_finished()`, where `e` is the target object of `*this`.

```
template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a);
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(f)` (C++Std [thread.decaycopy]).

Effects: `e.dispatch(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

```
template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a);
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(f)`.

Effects: `e.post(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

```
template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a);
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(f)`.

Effects: `e.defer(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

12.22.6. executor capacity

```
explicit operator bool() const noexcept;
```

Returns: `true` if `*this` has a target, otherwise `false`.

12.22.7. executor target access

const type_info& target_type() const noexcept;

Returns: If *this has a target of type T, typeid(T); otherwise, typeid(void).

template<class Executor> Executor* target() noexcept;

template<class Executor> const Executor* target() const noexcept;

Returns: If target_type() == typeid(Executor) a pointer to the stored executor target; otherwise a null pointer value.

12.22.8. executor comparisons

bool operator==(const executor& a, const executor& b) noexcept;

Returns:

- true if !a and !b;
- true if a and b share a target;
- true if e and f are the same type and e == f, where e is the target of a and f is the target of b;
- otherwise false.

bool operator==(const executor& e, nullptr_t) noexcept;

bool operator==(nullptr_t, const executor& e) noexcept;

Returns: !e.

bool operator!=(const executor& a, const executor& b) noexcept;

Returns: !(a == b).

bool operator!=(const executor& e, nullptr_t) noexcept;

bool operator!=(nullptr_t, const executor& e) noexcept;

Returns: (bool) e.

12.22.9. executor specialized algorithms

void swap(executor& a, executor& b) noexcept;

Effects: a.swap(b).

12.23. Function dispatch

template<class CompletionToken>

DEDUCED dispatch(CompletionToken&& token);

Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with `forward<CompletionToken>(token)`.
- Performs `ex.dispatch(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

Returns: `completion.result.get()`.

template<class Executor, class CompletionToken>

DEDUCED dispatch(const Executor& ex, CompletionToken&& token);

Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with `forward<CompletionToken>(token)`.
- Constructs a function object `f` containing as members:
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - an `executor_work_guard` object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`,and where the effect of `f()` is:
 - `w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by
 - `w.reset()`.

— Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`.

Returns: `completion.result.get()`.

Remarks: This function shall not participate in overload resolution unless `is_executor<Executor>::value` is true.

```
template<class ExecutionContext, class CompletionToken>
    DEDUCED dispatch(ExecutionContext& ctx, CompletionToken&& token);
```

Returns: `std::experimental::concurrency::dispatch(ctx.get_executor(), forward<CompletionToken>(token))`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t&>::value` is true.

12.24. Function post

[*Note:* The function post satisfies the requirements for an [asynchronous operation](#). —end note]

```
template<class CompletionToken>
    DEDUCED post(CompletionToken&& token);
```

Effects:

— Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with `forward<CompletionToken>(token)`.

— Performs `ex.post(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

Returns: `completion.result.get()`.

```
template<class Executor, class CompletionToken>
    DEDUCED post(const Executor& ex, CompletionToken&& token);
```

Effects:

— Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with `forward<CompletionToken>(token)`.

— Constructs a function object `f` containing as members:

- a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
- an `executor_work_guard` object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`,

and where the effect of `f()` is:

• `w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by

- `w.reset()`.

— Performs `ex.post(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`.

Returns: `completion.result.get()`.

Remarks: This function shall not participate in overload resolution unless `is_executor<Executor>::value` is true.

```
template<class ExecutionContext, class CompletionToken>
    DEDUCED post(ExecutionContext& ctx, CompletionToken&& token);
```

Returns: `std::experimental::concurrency::post(ctx.get_executor(), forward<CompletionToken>(token))`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t&>::value` is true.

12.25. Function defer

```
template<class CompletionToken>
    DEDUCED defer(CompletionToken&& token);
```

Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with `forward<CompletionToken>(token)`.
- Performs `ex.defer(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

Returns: `completion.result.get()`.

```
template<class Executor, class CompletionToken>
    DEDUCED defer(const Executor& ex, CompletionToken&& token);
```

Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with `forward<CompletionToken>(token)`.
- Constructs a function object `f` containing as members:
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - an `executor_work_guard` object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`,and where the effect of `f()` is:
 - `w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by
 - `w.reset()`.
- Performs `ex.defer(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`.

Returns: `completion.result.get()`.

Remarks: This function shall not participate in overload resolution unless `is_executor<Executor>::value` is `true`.

```
template<class ExecutionContext, class CompletionToken>
    DEDUCED defer(ExecutionContext& ctx, CompletionToken&& token);
```

Returns: `std::experimental::concurrency::defer(ctx.get_executor(), forward<CompletionToken>(token))`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is `true`.

12.26. Class template `strand`

The class template `strand` is a wrapper around an object of type `Executor` satisfying the [Executor requirements](#).

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class Executor>
    class strand
    {
    public:
        // types:

        typedef Executor inner_executor_type;

        // construct / copy / destroy:

        strand();
        explicit strand(Executor ex);
        template<class ProtoAllocator>
            strand(allocator_arg_t, const ProtoAllocator& alloc, Executor ex);
        strand(const strand& other) noexcept;
        strand(strand&& other) noexcept;
        template<class OtherExecutor> strand(const strand<OtherExecutor>& other) noexcept;
        template<class OtherExecutor> strand(strand<OtherExecutor>&& other) noexcept;

        strand& operator=(const strand& other) noexcept;
        strand& operator=(strand&& other) noexcept;
```

```

template<class OtherExecutor> strand& operator=(const strand<OtherExecutor>& other) noexcept;
template<class OtherExecutor> strand& operator=(strand<OtherExecutor>&& other) noexcept;

~strand();

// strand operations:

inner_executor_type get_inner_executor() const noexcept;

bool running_in_this_thread() const noexcept;

execution_context& context() noexcept;

void on_work_started() noexcept;
void on_work_finished() noexcept;

template<class Func, class ProtoAllocator>
    void dispatch(Func&& f, const ProtoAllocator& a);
template<class Func, class ProtoAllocator>
    void post(Func&& f, const ProtoAllocator& a);
template<class Func, class ProtoAllocator>
    void defer(Func&& f, const ProtoAllocator& a);

private:
    Executor inner_ex_; // exposition only
};

bool operator==(const strand<Executor>& a, const strand<Executor>& b);
bool operator!=(const strand<Executor>& a, const strand<Executor>& b);

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

strand<Executor> satisfies the [Executor](#) requirements.

A strand provides guarantees of ordering and non-concurrency. Given:

- strand objects s_1 and s_2 such that $s_1 == s_2$
- a function object f_1 added to the strand s_1 using post or defer, or using dispatch when $s_1.\text{running_in_this_thread}() == \text{false}$
- a function object f_2 added to the strand s_2 using post or defer, or using dispatch when $s_2.\text{running_in_this_thread}() == \text{false}$

then the implementation invokes f_1 and f_2 such that:

- the invocation of f_1 is not concurrent with the invocation of f_2
- the invocation of f_1 synchronizes with the invocation of f_2 .

Furthermore, if the addition of f_1 happens before the addition of f_2 , then the invocation of f_1 happens before the invocation of f_2 .

All member functions, except for the assignment operators and the destructor, do not introduce data races on `*this`, including its ordered, non-concurrent state. Additionally, constructors and assignment operators do not introduce data races on lvalue arguments.

If any function f executed by the strand throws an exception, the subsequent strand state is as if f had exited without throwing an exception.

12.26.1. strand constructors

```
strand();
```

Effects: Constructs an object of class `strand<Executor>` that represents a unique ordered, non-concurrent state. Initializes `inner_ex_` with `inner_ex_()`.

Remarks: This overload shall not participate in overload resolution unless `Executor` satisfies the

DefaultConstructible requirements (C++Std [defaultconstructible]).

```
explicit strand(Executor ex);
```

Effects: Constructs an object of class `strand<Executor>` that represents a unique ordered, non-concurrent state. Initializes `inner_ex_` as `inner_ex_(ex)`.

```
template<class ProtoAllocator>
strand(allocator_arg_t, const ProtoAllocator& a, Executor ex);
```

Effects: Constructs an object of class `strand<Executor>` that represents a unique ordered, non-concurrent state. Initializes `inner_ex_` as `inner_ex_(ex)`. A copy of the allocator argument `a` is used to allocate memory, if necessary, for the internal data structures of the constructed strand object.

```
strand(const strand& other) noexcept;
```

Effects: Initializes `inner_ex_` as `inner_ex_(other.inner_ex_)`.

Postconditions:

— `*this == other`

— `get_inner_executor() == other.get_inner_executor()`

```
strand(strand&& other) noexcept;
```

Effects: Initializes `inner_ex_` with `inner_ex_(std::move(other.inner_ex_))`.

Postconditions:

— `*this` is equal to the prior value of `other`

— `get_inner_executor() == other.get_inner_executor()`

```
template<class OtherExecutor> strand(const strand<OtherExecutor>& other) noexcept;
```

Requires: `OtherExecutor` is convertible to `Executor`.

Effects: Initializes `inner_ex_` with `inner_ex_(other.inner_ex_)`.

Postconditions: `*this == other`.

```
template<class OtherExecutor> strand(strand<OtherExecutor>&& other) noexcept;
```

Requires: `OtherExecutor` is convertible to `Executor`.

Effects: Initializes `inner_ex_` with `inner_ex_(std::move(other.inner_ex_))`.

Postconditions: `*this` is equal to the prior value of `other`.

12.26.2. strand assignment

```
strand& operator=(const strand& other) noexcept;
```

Requires: `Executor` is `CopyAssignable` (C++Std [copyassignable]).

Postconditions:

— `*this == other`

— `get_inner_executor() == other.get_inner_executor()`

Returns: `*this`.

```
strand& operator=(strand&& other) noexcept;
```

Requires: `Executor` is `MoveAssignable` (C++Std [moveassignable]).

Postconditions:

— `*this` is equal to the prior value of `other`

— `get_inner_executor() == other.get_inner_executor()`

Returns: `*this`.

```
template<class OtherExecutor> strand& operator=(const strand<OtherExecutor>& other) noexcept;
```

Requires: `OtherExecutor` is convertible to `Executor`. `Executor` is `CopyAssignable` (C++Std [copyassignable]).

Effects: Assigns `other.inner_ex_` to `inner_ex_`.

Postconditions: `*this == other`.

Returns: `*this`.

```
template<class OtherExecutor> strand& operator=(strand<OtherExecutor>&& other) noexcept;
```

Requires: `OtherExecutor` is convertible to `Executor`. `Executor` is `MoveAssignable` (C++Std [moveassignable]).

Effects: Assigns `std::move(other.inner_ex_)` to `inner_ex_`.

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

12.26.3. strand destructor

```
~strand();
```

Effects: Destroys an object of class `strand<Executor>`. After this destructor completes, objects that were added to the strand but have not yet been executed will be executed in a way that meets the guarantees of ordering and non-concurrency.

12.26.4. strand operations

```
inner_executor_type get_inner_executor() const noexcept;
```

Returns: `inner_ex_`.

```
bool running_in_this_thread() const noexcept;
```

Returns: `true` if the current thread of execution is running a function that was submitted to the strand, or to any other strand object `s` such that `s == *this`, using `dispatch`, `post` or `defer`; otherwise `false`. [Note: That is, the current thread of execution's call chain includes a function that was submitted to the strand. —end note]

```
execution_context& context() noexcept;
```

Returns: `inner_ex_.context()`.

```
void on_work_started() noexcept;
```

Effects: Calls `inner_ex_.on_work_started()`.

```
void on_work_finished() noexcept;
```

Effects: Calls `inner_ex_.on_work_finished()`.

```
template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a);
```

Effects: If `running_in_this_thread() == true`, calls `DECAY_COPY(forward<Func>(f))()` (C++Std [thread.decaycopy]). [Note: If `f` exits via an exception, the exception propagates to the caller of `dispatch()`. —end note] Otherwise, requests invocation of `f`, as if by forwarding the function object `f` and allocator `a` to the executor `inner_ex_`, such that the guarantees of ordering and non-concurrency are met.

```
template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a);
```

Effects: Requests invocation of `f`, as if by forwarding the function object `f` and allocator `a` to the executor `inner_ex_`, such that the guarantees of ordering and non-concurrency are met.

```
template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a);
```

Effects: Requests invocation of `f`, as if by forwarding the function object `f` and allocator `a` to the executor `inner_ex_`, such that the guarantees of ordering and non-concurrency are met.

12.26.5. strand comparisons

```
bool operator==(const strand<Executor>& a, const strand<Executor>& b);
```

Returns: true, if the strand objects share the same ordered, non-concurrent state; otherwise false.

```
bool operator!=(const strand<Executor>& a, const strand<Executor>& b);
```

Returns: !(a == b).

12.27. Class template use_future_t

The class template use_future_t defines a set of types that, when passed as a [completion token](#) to an asynchronous operation's initiating function, cause the result of the asynchronous operation to be delivered via a future (C++Std [futures.unique_future]).

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

template<class ProtoAllocator = allocator<void>>
class use_future_t
{
public:
    // use_future_t types:
    typedef ProtoAllocator allocator_type;

    // use_future_t members:
    constexpr use_future_t() noexcept;
    explicit use_future_t(const allocator_type& a) noexcept;
    template<class OtherProtoAllocator> use_future_t<OtherProtoAllocator>
        rebind(const OtherProtoAllocator& a) const noexcept;
    allocator_type get_allocator() const noexcept;
    template <class F> unspecified operator()(F&& f) const;
};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

12.27.1. use_future_t constructors

```
constexpr use_future_t() noexcept;
```

Effects: Constructs a use_future_t with a default-constructed allocator.

```
explicit use_future_t(const allocator_type& a) noexcept;
```

Postconditions: get_allocator() == a.

12.27.2. use_future_t members

```
template<class OtherProtoAllocator> use_future_t<OtherProtoAllocator>
rebind(const OtherProtoAllocator& a) const noexcept;
```

Returns: A use_future_t object where get_allocator() == a.

```
allocator_type get_allocator() const noexcept;
```

Returns: The associated allocator object.

```
template <class F> unspecified operator()(F&& f) const;
```

Let T be a completion token type. Let H be a completion handler type and let h be an object of type H . Let FD be the type $\text{decay_t}<F>$ and let fd be an lvalue of type FD constructed with $\text{forward}<F>(f)$. Let $\text{Args} \dots$ be the completion signature of H and let N be $\text{sizeof} \dots (\text{Args})$. Let i be in the range $[0, N)$ and let A_i be the i th type in Args . Let a_i be the i th argument associated with A_i .

Returns: A completion token t of type T .

Remarks: The return type T satisfies the Destructible (C++Std [destructible]) and MoveConstructible (C++Std [moveconstructible]) requirements.

The object h of type H is an asynchronous provider with an associated shared state (C++Std [futures.state]).

The effect of $h(a_0, \dots, a_{N-1})$ is to atomically store the result of `INVOKE(f, forward<A0>(a0), ..., forward<AN-1>(aN-1))` (C++Std [func.require]) in the shared state and make the shared state ready. If `f` exits via an exception then that exception is atomically stored in the shared state and the shared state is made ready.

The implementation provides a partial specialization template `<class Result, class... Args> async_result<T, Result(Args...)>` such that:

- the nested typedef `completion_handler_type` is a type `H`;
- the nested typedef `return_type` is `future<result_of_t<FD(decay_t<Args>...)>>`; and
- when an object `r1` of type `async_result<T, Result(Args...)>` is constructed from `h`, the expression `r1.get()` returns a future with the same shared state as `h`.

For any executor type `E`, the associated object for the associator `associated_executor<H, E>` is an executor where, for function objects executed using the executor's `dispatch()`, `post()` or `defer()` functions, any exception thrown is caught by a function object and stored in the associated shared state.

12.27.3. Partial class template specialization `async_result` for `use_future_t`

```
template<class ProtoAllocator, class Result, class... Args>
class async_result<use_future_t<ProtoAllocator>, Result(Args...)>
{
    typedef see below completion_handler_type;
    typedef see below return_type;

    explicit async_result(completion_handler_type& h);
    async_result(const async_result&) = delete;
    async_result& operator=(const async_result&) = delete;

    return_type get();
};
```

Let `R` be the type `async_result<use_future_t<ProtoAllocator>, Result(Args...)>`. Let `F` be the nested function object type `R::completion_handler_type`.

An object `t1` of type `F` is an asynchronous provider with an associated shared state (C++Std [futures.state]). The type `F` provides `F::operator()` such that the expression `t1(declval<Args>()...)` is well formed.

The implementation specializes `associated_executor` for `F`. For function objects executed using the associated executor's `dispatch()`, `post()` or `defer()` functions, any exception thrown is caught by the executor and stored in the associated shared state.

For any executor type `E`, the associated object for the associator `associated_executor<F, E>` is an executor where, for function objects executed using the executor's `dispatch()`, `post()` or `defer()` functions, any exception thrown by a function object is caught by the executor and stored in the associated shared state.

When an object `r1` of type `R` is constructed from `t1`, the expression `r1.get()` returns a future with the same shared state as `t1`.

The type of `R::return_type` and the effects of `F::operator()` are defined in the table below. After establishing these effects, `F::operator()` makes the shared state ready. In this table, `N` is the value of `sizeof... (Args)`; let i be in the range $[0, N)$ and let T_i be the i th type in `Args`; let U_i be `decay_t<Ti>` for each type T_i in `Args`; let A_i be the deduced type of the i th argument to `F::operator()`; and let a_i be the i th argument to `F::operator()`.

Table 9. `async_result<use_future_t<ProtoAllocator>, Result(Args...)>` semantics

N	U_0	$R::\text{return_type}$	$F::\text{operator}()$ effects
0		<code>future<void></code>	None.
1	<code>error_code</code>	<code>future<void></code>	If a_0 evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a₀))</code> in the shared state.
1	<code>exception_ptr</code>	<code>future<void></code>	If a_0 is non-null, atomically stores the exception pointer a_0 in the shared state.

1	all other types	future<U ₀ >	Atomically stores forward<A ₀ >(a ₀) in the shared state.
2	error_code	future<U ₁ >	If a ₀ evaluates to true, atomically stores the exception pointer produced by make_exception_ptr(system_error(a ₀)) in the shared state; otherwise, atomically stores forward<A ₁ >(a ₁) in the shared state.
2	exception_ptr	future<U ₁ >	If a ₀ is non-null, atomically stores the exception pointer in the shared state; otherwise, atomically stores forward<A ₁ >(a ₁) in the shared state.
2	all other types	future<tuple<U ₀ , U ₁ >>	Atomically stores forward_as_tuple<A ₀ , A ₁ >(a ₀ , a ₁) in the shared state.
>2	error_code	future<tuple<U ₁ , ..., U _{N-1} >>	If a ₀ evaluates to true, atomically stores the exception pointer produced by make_exception_ptr(system_error(a ₀)) in the shared state; otherwise, atomically stores forward_as_tuple<A ₁ , ..., A _{N-1} >(a ₁ , ..., a _{N-1}) in the shared state.
>2	exception_ptr	future<tuple<U ₁ , ..., U _{N-1} >>	If a ₀ is non-null, atomically stores the exception pointer in the shared state; otherwise, atomically stores forward_as_tuple<A ₁ , ..., A _{N-1} >(a ₁ , ..., a _{N-1}) in the shared state.
>2	all other types	future<tuple<U ₀ , ..., U _{N-1} >>	Atomically stores forward_as_tuple<A ₀ , ..., A _{N-1} >(a ₀ , ..., a _{N-1}) in the shared state.

12.28. Partial class template specialization `async_result` for `packaged_task`

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class Result, class... Args, class Signature>
    class async_result<packaged_task<Result(Args...)>, Signature>
    {
    public:
        typedef packaged_task<Result(Args...)> completion_handler_type;
        typedef future<Result> return_type;

        explicit async_result(completion_handler_type& h);
        async_result(const async_result&) = delete;
        async_result& operator=(const async_result&) = delete;

        return_type get();

    private:
        return_type future_; // exposition only
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

explicit async_result(completion_handler_type& h);

Effects: Initializes future_ with h.get_future().

return_type get();

Returns: std::move(future_).

12.29. Header `<experimental/thread_pool>` synopsis

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

```

```

class thread\_pool;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

12.30. Class `thread_pool`

Class `thread_pool` implements a fixed-size pool of threads.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread_pool : public execution_context
    {
    public:
        // types:

        class executor\_type;

        // construct / copy / destroy:

        thread_pool();
        explicit thread_pool(std::size_t num_threads);
        thread_pool(const thread_pool&) = delete;
        thread_pool& operator=(const thread_pool&) = delete;
        ~thread_pool();

        // thread_pool operations:

        executor_type get_executor() noexcept;

        void stop();

        void join();
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The class `thread_pool` satisfies the [ExecutionContext type requirements](#).

For an object of type `thread_pool`, *outstanding work* is defined as the sum of:

- the total number of calls to the `on_work_started` function, less the total number of calls to the `on_work_finished` function, to any executor of the `thread_pool`.
- the number of function objects that have been added to the `thread_pool` via the `thread_pool` executor, but not yet executed; and
- the number of function objects that are currently being executed by the `thread_pool`.

The `thread_pool` member functions `get_executor`, `stop`, and `join`, and the `thread_pool::executor_type` copy constructors, member functions and comparison operators, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

12.30.1. `thread_pool` members

```

thread_pool();
explicit thread_pool(std::size_t num_threads);

```

Effects: Creates an object of class `thread_pool` containing a number of threads of execution, each represented by a thread object. If specified, the number of threads in the pool is `num_threads`. Otherwise, the number of threads in the pool is implementation-defined. [Note: A suggested value for the implementation-defined number of threads is `std::thread::hardware_concurrency() * 2`.—*end note*]

```

~thread_pool();

```

Effects: Destroys an object of class `thread_pool`. Performs `stop()` followed by `join()`.

```
executor_type get_executor() noexcept;
```

Returns: An executor that may be used for submitting function objects to the `thread_pool`.

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete.

```
void join();
```

Effects: If not already stopped, signals the threads in the pool to exit once the outstanding work is 0. Blocks the calling thread (C++Std [defns.block]) until all threads in the pool have completed.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `join()` return.

12.31. Class `thread_pool::executor_type`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread_pool::executor_type
    {
    public:
        // construct / copy / destroy:

        executor_type(const executor_type& other) noexcept;
        executor_type(executor_type&& other) noexcept;

        executor_type& operator=(const executor_type& other) noexcept;
        executor_type& operator=(executor_type&& other) noexcept;

        // executor operations:

        bool running_in_this_thread() const noexcept;

        thread_pool& context() noexcept;

        void on_work_started() noexcept;
        void on_work_finished() noexcept;

        template<class Func, class ProtoAllocator>
            void dispatch(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void post(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void defer(Func&& f, const ProtoAllocator& a);
    };

    bool operator==(const thread_pool::executor_type& a,
                    const thread_pool::executor_type& b) noexcept;
    bool operator!=(const thread_pool::executor_type& a,
                    const thread_pool::executor_type& b) noexcept;

    template<> struct is_executor<thread_pool::executor_type> : true_type {};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

`thread_pool::executor_type` is a type satisfying [Executor requirements](#). Objects of type `thread_pool::executor_type` are associated with a `thread_pool`, and function objects submitted using the `dispatch`, `post`, or `defer` member functions will be executed by the `thread_pool`.]

12.31.1. `thread_pool::executor_type` constructors

executor_type(const executor_type& other) noexcept;

Postconditions: *this == other.

executor_type(executor_type&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

12.31.2. thread_pool::executor_type assignment

executor_type& operator=(const executor_type& other) noexcept;

Postconditions: *this == other.

Returns: *this.

executor_type& operator=(executor_type&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

Returns: *this.

12.31.3. thread_pool::executor_type operations

bool running_in_this_thread() const noexcept;

Returns: true if the current thread of execution is calling a run function of the associated thread_pool object.

[*Note:* That is, the current thread of execution's call chain includes a run function. —*end note*]

thread_pool& context() noexcept;

Returns: A reference to the associated thread_pool object.

void on_work_started() noexcept;

Effects: Increment the count of outstanding work associated with the thread_pool.

void on_work_finished() noexcept;

Effects: Decrement the count of outstanding work associated with the thread_pool.

template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a);

Effects: If running_in_this_thread() is true, calls *DECAY_COPY*(forward<Func>(f))() (C++Std [thread.decaycopy]). [*Note:* If f exits via an exception, the exception propagates to the caller of dispatch(). —*end note*] Otherwise, calls post(forward<Func>(f), a).

template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a);

Effects: Adds f to the thread_pool.

template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a);

Effects: Adds f to the thread_pool.

12.31.4. thread_pool::executor_type comparisons

bool operator==(const thread_pool::executor_type& a,
const thread_pool::executor_type& b) noexcept;

Returns: addressof(a.context()) == addressof(b.context()).

bool operator!=(const thread_pool::executor_type& a,
const thread_pool::executor_type& b) noexcept;

Returns: !(a == b).

12.32. Header <experimental/loop_scheduler> synopsis

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class loop\_scheduler;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

12.33. Class `loop_scheduler`

Class `loop_scheduler` implements a pool of threads where existing threads are assigned to the pool by the program.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class loop_scheduler : public execution_context
    {
    public:
        // types:

        class executor\_type;
        typedef implementation defined count_type;

        // construct / copy / destroy:

        loop_scheduler();
        explicit loop_scheduler(int concurrency_hint);
        loop_scheduler(const loop_scheduler&) = delete;
        loop_scheduler& operator=(const loop_scheduler&) = delete;

        // loop_scheduler operations:

        executor_type get_executor() noexcept;

        count_type run();
        template<class Rep, class Period>
            count_type run_for(const chrono::duration<Rep, Period>& rel_time);
        template<class Clock, class Duration>
            count_type run_until(const chrono::time_point<Clock, Duration>& abs_time);

        count_type run_one();
        template<class Rep, class Period>
            count_type run_one_for(const chrono::duration<Rep, Period>& rel_time);
        template<class Clock, class Duration>
            count_type run_one_until(const chrono::time_point<Clock, Duration>& abs_time);

        count_type poll();

        count_type poll_one();

        void stop();

        bool stopped() const noexcept;

        void restart();
    };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The class `loop_scheduler` satisfies the [ExecutionContext type requirements](#).

`count_type` is an implementation-defined unsigned integral type of at least 32 bits.

The `loop_scheduler` member functions `run`, `run_for`, `run_until`, `run_one`, `run_one_for`, `run_one_until`, `poll`, and `poll_one` are collectively referred to as the *run functions*.

For an object of type `loop_scheduler`, *outstanding work* is defined as the sum of:

- the total number of calls to the `on_work_started` function, less the total number of calls to the `on_work_finished` function, to any executor of the `loop_scheduler`.
- the number of function objects that have been added to the `loop_scheduler` via any executor of the `loop_scheduler`, but not yet executed; and
- the number of function objects that are currently being executed by the `loop_scheduler`.

If at any time the outstanding work falls to 0, the `loop_scheduler` is stopped as if by `stop()`.

The `loop_scheduler` member functions `get_executor`, `stop`, and `stopped`, the `run` functions, and the `loop_scheduler::executor_type` copy constructors, member functions and comparison operators, do not introduce data races as a result of concurrent calls to those functions from different threads of execution. [*Note: The `restart` member function is excluded from these thread safety requirements. —end note*]

12.33.1. `loop_scheduler` members

```
loop_scheduler();  
explicit loop_scheduler(int concurrency_hint);
```

Effects: Creates an object of class `loop_scheduler`.

Remarks: The `concurrency_hint` parameter is a suggestion to the implementation on the number of threads that should execute function objects.

```
executor_type get_executor() noexcept;
```

Returns: An executor that may be used for submitting function objects to the `loop_scheduler`.

```
count_type run();
```

Requires: Must not be called from a thread that is currently calling a `run` function.

Effects: Equivalent to:

```
count_type n = 0;  
while (run_one())  
    if (n != numeric_limits<count_type>::max())  
        ++n;
```

Returns: `n`.

```
template<class Rep, class Period>  
count_type run_for(const chrono::duration<Rep, Period>& rel_time);
```

Effects: Equivalent to:

```
return run_until(chrono::steady_clock::now() + rel_time);
```

```
template<class Clock, class Duration>  
count_type run_until(const chrono::time_point<Clock, Duration>& abs_time);
```

Effects: Equivalent to:

```
count_type n = 0;  
while (run_one_until(abs_time))  
    if (n != numeric_limits<count_type>::max())  
        ++n;
```

Returns: `n`.

```
count_type run_one();
```

Requires: Must not be called from a thread that is currently calling a `run` function.

Effects: If the `loop_scheduler` object has no outstanding work, performs `stop()`. Otherwise, blocks while the `loop_scheduler` has outstanding work, or until the `loop_scheduler` is stopped, or until one function object has been executed.

If an executed function object throws an exception, the exception propagates to the caller of `run_one()`. The

loop_scheduler state is as if the function object had returned normally.

Returns: 1 if a function object was executed, otherwise 0.

Notes: This function may invoke additional function objects through nested calls to the loop_scheduler executor's dispatch member function. These do not count towards the return value.

```
template<class Rep, class Period>
count_type run_one_for(const chrono::duration<Rep, Period>& rel_time);
```

Effects: Equivalent to:

```
return run_one_until(chrono::steady_clock::now() + rel_time);
```

```
template<class Clock, class Duration>
count_type run_one_until(const chrono::time_point<Clock, Duration>& abs_time);
```

Effects: If the loop_scheduler object has no outstanding work, performs stop(). Otherwise, blocks while the loop_scheduler has outstanding work, or until the expiration of the absolute timeout (C++Std [thread.req.timing]) specified by abs_time, or until the loop_scheduler is stopped, or until one function object has been executed.

If an executed function object throws an exception, the exception propagates to the caller of run_one(). The loop_scheduler state is as if the function object had returned normally.

Returns: 1 if a function object was executed, otherwise 0.

Notes: This function may invoke additional function objects through nested calls to the loop_scheduler executor's dispatch member function. These do not count towards the return value.

```
count_type poll();
```

Effects: Equivalent to:

```
count_type n = 0;
while (poll_one())
    if (n != numeric_limits<count_type>::max())
        ++n;
```

Returns: n.

```
count_type poll_one();
```

Effects: If the loop_scheduler object has no outstanding work, performs stop(). Otherwise, if there is a function object ready for immediate execution, executes it.

If an executed function object throws an exception, the exception propagates to the caller of poll_one(). The loop_scheduler state is as if the function object had returned normally.

Returns: 1 if a function object was invoked, otherwise 0.

Notes: This function may invoke additional function objects through nested calls to the loop_scheduler executor's dispatch member function. These do not count towards the return value.

```
void stop();
```

Effects: Stops the loop_scheduler. Concurrent calls to any run function will end as soon as possible. If a call to a run function is currently executing a function object, the call will end only after completion of that function object. The call to stop() returns without waiting for concurrent calls to run functions to complete.

Postconditions: stopped() == true.

[*Note:* When stopped() == true, subsequent calls to a run function will exit immediately with a return value of 0, without executing any function objects. A loop_scheduler remains in the stopped state until a call to restart().—end note]

```
bool stopped() const noexcept;
```

Returns: true if the loop_scheduler is stopped.

```
void restart();
```

Postconditions: stopped() == false.

12.34. Class `loop_scheduler::executor_type`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class loop_scheduler::executor_type
    {
    public:
        // construct / copy / destroy:

        executor_type(const executor_type& other) noexcept;
        executor_type(executor_type&& other) noexcept;

        executor_type& operator=(const executor_type& other) noexcept;
        executor_type& operator=(executor_type&& other) noexcept;

        // executor operations:

        bool running_in_this_thread() const noexcept;

        loop_scheduler& context() noexcept;

        void on_work_started() noexcept;
        void on_work_finished() noexcept;

        template<class Func, class ProtoAllocator>
            void dispatch(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void post(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void defer(Func&& f, const ProtoAllocator& a);
    };

    bool operator==(const loop_scheduler::executor_type& a,
                    const loop_scheduler::executor_type& b) noexcept;
    bool operator!=(const loop_scheduler::executor_type& a,
                    const loop_scheduler::executor_type& b) noexcept;

    template<> struct is_executor<loop_scheduler::executor_type> : true_type {};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

`loop_scheduler::executor_type` is a type satisfying [Executor requirements](#). Objects of type `loop_scheduler::executor_type` are associated with a `loop_scheduler`, and function objects submitted using the `dispatch`, `post`, or `defer` member functions will be executed by the `loop_scheduler` from within a `run` function.]

12.34.1. `loop_scheduler::executor_type` constructors

```
executor_type(const executor_type& other) noexcept;
```

Postconditions: `*this == other`.

```
executor_type(executor_type&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

12.34.2. `loop_scheduler::executor_type` assignment

```
executor_type& operator=(const executor_type& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
executor_type& operator=(executor_type&& other) noexcept;
```

Postconditions: *this is equal to the prior value of other.

Returns: *this.

12.34.3. loop_scheduler::executor_type operations

```
bool running_in_this_thread() const noexcept;
```

Returns: true if the current thread of execution is calling a run function of the associated loop_scheduler object. [Note: That is, the current thread of execution's call chain includes a run function. —end note]

```
loop_scheduler& context() noexcept;
```

Returns: A reference to the associated loop_scheduler object.

```
void on_work_started() noexcept;
```

Effects: Increment the count of outstanding work associated with the loop_scheduler.

```
void on_work_finished() noexcept;
```

Effects: Decrement the count of outstanding work associated with the loop_scheduler.

```
template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a);
```

Effects: If running_in_this_thread() is true, calls `DECAY_COPY(forward<Func>(f))()` (C++Std [thread.decaycopy]). [Note: If f exits via an exception, the exception propagates to the caller of dispatch(). —end note] Otherwise, calls `post(forward<Func>(f), a)`.

```
template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a);
```

Effects: Adds f to the loop_scheduler.

```
template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a);
```

Effects: Adds f to the loop_scheduler.

12.34.4. loop_scheduler::executor_type comparisons

```
bool operator==(const loop_scheduler::executor_type& a,
                const loop_scheduler::executor_type& b) noexcept;
```

Returns: `addressof(a.context()) == addressof(b.context())`.

```
bool operator!=(const loop_scheduler::executor_type& a,
                const loop_scheduler::executor_type& b) noexcept;
```

Returns: `!(a == b)`.

13. Acknowledgements

The author would like to thank Jamie Allsop, Arash Partow and Dietmar Kühl for providing feedback, corrections and suggestions on early iterations of both the library implementation and this proposal.

The author would also like to thank Marshall Clow, Jens Maurer, Arash Partow, Jamie Allsop, Dietmar Kühl, Detlef Vollmann, Jonathan Wakely, Mikael Kilpeläinen, Jens Weller, Michael Wong, Eric Fisselier, and Jeffrey Yasskin for participating in the Cologne LWG wording review of the Networking Library TS proposal, which contributed significantly to the changes in this revision.

[1] The concept itself is not new, however the term is taken from [Holgate, Len, *Activatable Object*, ACCU Overload Journal #122, August 2014](#)

[2] <http://theron-library.com>

